Journal of Bionic Engineering (2024) 21:2658–2684
https://doi.org/10.1007/s42235-024-00575-7

RESEARCH ARTICLE

# An Intrusion Detection System on The Internet of Things Using Deep Learning and Multi-objective Enhanced Gorilla Troops Optimizer

**Hossein Asgharzadeh[1,2] · Ali Ghaffari[3,4,5] · Mohammad Masdari[1] · Farhad Soleimanian Gharehchopogh[1]**

Received: 15 November 2022 / Revised: 14 June 2024 / Accepted: 18 June 2024 / Published online: 9 July 2024
© The Author(s) 2024

## Abstract

In recent years, developed Intrusion Detection Systems (IDSs) perform a vital function in improving security and anomaly detection. The effectiveness of deep learning-based methods has been proven in extracting better features and more accurate classification than other methods. In this paper, a feature extraction with convolutional neural network on Internet of Things (IoT) called FECNNIoT is designed and implemented to better detect anomalies on the IoT. Also, a binary multi-objective enhance of the Gorilla troops optimizer called BMEGTO is developed for effective feature selection. Finally, the combination of FECNNIoT and BMEGTO and KNN algorithm-based classification technique has led to the presentation of a hybrid method called CNN-BMEGTO-KNN. In the next step, the proposed model is implemented on two benchmark data sets, NSL-KDD and TON-IoT and tested regarding the accuracy, precision, recall, and F1-score criteria. The proposed CNN-BMEGTO-KNN model has reached 99.99% and 99.86% accuracy on TON-IoT and NSL-KDD datasets, respectively. In addition, the proposed BMEGTO method can identify about 27% and 25% of the effective features of the NSL-KDD and TON-IoT datasets, respectively.

**Keywords** Intrusion detection · Internet of Things · Convolutional neural network · Multi-objective · Gorilla troops optimizer

## 1 Introduction

In recent years, IoT has attracted the interest of many researchers due to its wide applications in various fields such as smart homes, smart cities, and electronic health [1, 2]. On the other hand, many individuals, businesses, and organizations depend on Internet-based services to fulfill their commonplace activities, and numerous important infrastructures are composed of IoT devices [3, 4].

The most important challenge of the IoT platform is the vulnerability of devices to various types of cyber-attacks due to limited resources [5–7] and the growing number of IoT devices and diverse communication protocols [8–11]. One of the negative results of this expansion is the violation of the security of the nodes, the increased attacks, and espionage missions [12]. Because security challenges are the biggest problem raised in this technology, establishing secure communication should be highly followed on the IoT. Therefore, designing and implementing anomaly detection systems in this network and its devices is considered a vital process to reduce attacks and establish security [13].

✉ Ali Ghaffari
A.Ghaffari@iaut.ac.ir

Hossein Asgharzadeh
h.asgharzadeh66@yahoo.com

Farhad Soleimanian Gharehchopogh
bonab.farhad@gmail.com

1 Department of Computer Engineering, Urmia Branch, Islamic Azad University, Urmia 63896-57169, Iran

2 Department of Computer Engineering, Technical and Vocational University (TUV), Tehran, Iran

3 Department of Computer Engineering, Tabriz Branch, Islamic Azad University, Tabriz 5157944533, Iran

4 Department of Computer Engineering, Faculty of Engineering and Natural Science, Istinye University, 34010 Istanbul, Turkey

5 Department of Computer Engineering, Khazar University, AZ1096 Baku, Azerbaijan

Intrusion detection systems facilitate identifying anomalies in IoT networks and can take steps to get a secure and reliable environment [14, 15]. In the last decade, intrusion and anomaly detection systems based on machine learning and deep learning methods have been developed. Machine learning methods such as decision trees, support vector machines, random forests, etc., have been studied by various researchers to diagnose anomalies [16, 17]. Detection of attacks and anomalies on the Internet of Things using machine learning approaches such as Artificial Neural Network (ANN), Support Vector Machine (SVM), Logistic Regression (LR), Random Forest (RF), and Decision Tree (DT) on dataset DS2OS-Kaggle was realized in Refs. [18, 19].

Some researchers have used feature selection methods based on meta-heuristic algorithms to reduce features in high-dimensional datasets [20, 21]. A new approach to classification-based feature selection using the enhanced whale optimization algorithm for detecting IoT attacks and managing high-dimensional problems has been proposed [22, 23]. Although machine learning methods are very successful in detecting anomalies, increasing features and data volume have to some extent reduced their efficiency [24–26]. On the other hand, the results of various research prove that deep learning approaches have performed better than machine learning methods, especially on large data sets. As a result, due to multiple systems generating a significant amount of data with high-dimensional features, deep learning methods are suitable for developing IoT intrusion detection systems.

Deep learning is an advanced sub-branch of machine learning containing several layers of neurons to realize the learning process. Unlike machine learning methods, deep learning can adapt its performance to macro-scale data. Deep learning techniques have proven their effectiveness in dimensionality reduction, feature extraction, and data classification. In deep learning-based intrusion and anomaly detection systems, deep networks learn from historical traffic data and classify data behavior into two classes, normal and attack. Some researchers have developed anomaly detection systems based on deep belief networks in the IoT. Deep recurrent networks have received much attention in detecting time-based anomalies on the Internet of Things.

A convolution neural network is another prominent contribution of the deep network in detecting anomalies in the IoT. The rationale for this effect is the ability to automatically extract low and high-level features from different data and compute faster in the convolutional neural network. Convolution neural networks are great alternatives for detecting and classifying anomalies because they can automatically classify key features and conduct computations faster.

The Gorilla Troops optimizer is one of the new meta-heuristic optimization algorithms [27]. This method is a single-objective and continuous algorithm inspired by the behavior of Gorillas seeking food in the forest. It is used to select important features in the data set. As a result, it is necessary to make fundamental changes to this algorithm to optimize its use in feature selection. In short, the contribution presented in this paper can be summarized as follows:

- Developing a hybrid method with multi-objective optimization algorithm and neural network to increase the accuracy of anomaly detection in IoT.
- Designing an improved binary version of the gorilla algorithm in multi-objective with the aim of selecting the best features.
- Designing an advanced convolutional neural network to extract high-level and low-level features with the aim of increasing the accuracy of anomaly detection in the IoT.

The rest of the paper is organized as follows. The second section examines the work related to intrusion detection and feature selection. The basic concepts of the Gorilla Troops optimizer are explained in the third section. An anomaly detection method on the Internet of Things based on a Convolutional Neural Network and Multi-Objective Gorilla Troops optimizer named CNN-BMEGTO-KNN is presented in the fourth section. The evaluation of the proposed method and its comparison with other deep and machine learning algorithms are presented in the fifth section. Finally, the conclusion and research direction for future works are stated in the last section.

## 2 Related Work

This section has reviewed outstanding research in anomaly detection and feature selection based on machine learning and deep learning methods. A summary of previous work is presented at the end of the section in Table 1. Below are more details of each work.

In Ref. [28], the combination of GWO and PSO algorithms is proposed to extract important features in the Internet of Things network. The proposed method is tested on three datasets KDDCup99, NSL-KDD, and CICIDS-2017. The results of various experiments showed that the proposed

**Table 1** Previous works related to intrusion detection using feature selection (FS) and deep learning

| Reference | Year | Data set | Detection and feature selection method | Description |
|---|---|---|---|---|
| [28] | 2021 | KDD99<br>NSL-KDD CICIDS2017 | Classification algorithm: Random Forest<br>FS: Single objective – meta heuristic | In this article, a combination of GWO and single-objective PSO algorithm is used for feature selection, and finally, forest algorithm is considered for classification |
| [29] | 2021 | – | Classification algorithm: K-nearest neighbor algorithm<br>FS: Single-objective –Enhanced whale optimization algorithm | In this article, the k-nearest neighbor algorithm for classification and for feature selection of the enhanced whale optimizer algorithm is used. But deep learning methods have not been used |
| [30] | 2020 | UNSW Bot-IoT | Classification algorithm: Convolutional neural network<br>FS: choose the best feature | Anomaly detection in the Internet of Things is based on a vector convolutional neural network called VCDL and uses selected features based on the data set (top 10 features) |
| [31] | 2021 | BoT-IoT<br>CICIDS2017 | Classification algorithm: Convolutional neural network<br>FS: – | The detection method with deep feedforward neural network is presented to increase the accuracy of abnormality detection. A smaller dataset is used in this article |
| [32] | 2020 | NSL-KDD CICIDS2017 | Classification algorithm:A combination of machine learning methods<br>FS:– | In this article, a new approach based on the combination of deep neural network and k nearest neighbor has been used, which has increased the accuracy in anomaly detection. But for the NSL-KDD dataset, no special feature selection method is considered |
| [33] | 2020 | NSL-KDD<br>UNSW—NB15 | Classification algorithm: Artificial neural network<br>FS: Multi-objective—Improved multi-objective immune system | In this paper, artificial neural network and improved multi-objective immune system for anomaly detection are presented. But different objectives for feature selection, precision, accuracy, F1 have not been investigated at the same time |
| [33] | 2020 | CICIDS2017 | Classification algorithm: Extreme machine learning<br>FS: Multi-objective—multi-objective based on NSGA-II | In this paper, a multi-objective feature selection method based on NSGA-II is designed. But neural networks have not been used to extract local and global features. Also, the proposed method has been evaluated only on one data set |
| [20] | 2020 | KDD99<br>NSL-KDD<br>UNSW—NB15 | Classification algorithm: Machine learning methods<br>FS: Single-objective—Pigeon-inspired optimizer algorithm | In this article, the combination of machine learning methods and single-objective pigeon optimizer algorithm is designed for feature selection, which was able to select important features and increase the accuracy to some extent |
| [36] | 2021 | KDD99<br>NSL-KDD CICIDS2017<br>BoT-IoT | Classification algorithm: Convolutional neural network<br>FS: Single objective—Advanced transient search optimization algorithm | A hybrid method based on convolutional neural network and advanced transient search optimization algorithm feature selection has been designed, which has increased accuracy and speed in anomaly detection. But single-objective methods do not consider multiple feature selection criteria |
| [37] | 2021 | – | Classification algorithm: Convolutional neural network and particle swarm optimization<br>FS:– | The combination of convolutional neural network and particle swarm optimization is presented to increase the accuracy of anomaly detection. But in this paper, a smaller data set is used |
| [38] | 2021 | TON-IoT | Classification algorithm: DNN deep learning and ensemble methods such as GBM<br>FS: – | In this article, a new data set named "TON-IOT" is introduced and also deep learning DNN and GBM method are used to test on this data set |
| [39] | 2021 | TON-IoT | Classification algorithm: Deep learning method<br>FS: – | In this paper, an intelligent agent based on a neural network method called Swarm-NN (Swarm-NN) is proposed to identify attackers in the framework of IoMT |

**Table 1** (continued)

| Reference | Year | Data set | Detection and feature selection method | Description |
|---|---|---|---|---|
| Proposed scheme | 2022 | NSL-KDD<br>TON-IoT | Classification algorithm: FECNNIoT-KNN<br>FS: enhanced multi objective—BMEGTO | In this article, advanced convolutional neural network is used to extract local and global features. An improved feature selection algorithm with multiple feature selection criteria called BMEGTO is also presented. In addition, two valid and important new datasets have been used to detect anomalies in the Internet of Things |

model and the random forest have achieved an accuracy of 99.66%, 99.24%, and 99.88% and showed superior performance compared to LSTM-RNN, ANN, GRU-RNN, and BRNN methods. In Ref. [29], V-shape and s-shape transfer functions are used for binarization. As well as a variety of binarization selection methods are to strengthen the whale optimization algorithm. Finally, the proposed method is implemented on the Bot-IoT dataset. Various experiments show that the whale optimization algorithm based on the V-shape transfer function combined with the elitist tournament binarization method is superior to the s-shape transfer function.

The proposed neural network in Ref. [30] consists of several layers of convolution, max-pooling, and full connection layer. The proposed model named VCDL with 10 extracted features is applied to the UNSW Bot-IoT dataset and achieved an accuracy of 99.9985%. Also, recurrent neural networks and multilayer perceptron have achieved 99.998% and 99.997% accuracy, respectively. In Ref. [31] uses a convolutional neural network to create a multi-class classification model. Then the proposed model based on convolutional neural networks is implemented in multiple dimensions. In addition, transfer learning is used to implement pre-trained binary and multi-class convolutional neural network models. Finally, the proposed method was applied to the BoT-IoT and CICIDS-2017 datasets and achieved an accuracy of 99%.

In Ref. [32], a two-phase method, including binary and multi-class detection based on a two-layer deep neural network, is designed. The proposed method is implemented on NSL-KDD and CICIDS2017 datasets and has achieved 99.7% and 99.8% accuracy, respectively, which has performed better than other methods. In Ref. [33], an improved multi-objective immune system algorithm based on a non-dominated neighbor is used to select meaningful features. An artificial neural network is used to detect or classify intrusion. Finally, the proposed approach MOIA is implemented on two datasets, NSL-KDD and UNSW-NB15. The results of various tests proved that the proposed approach had achieved 99.47% and 79.81% accuracy, respectively, which has obtained a lower number of features and better accuracy than the I-NSGA-III and NSGA-II algorithms. In Ref. [34], an extreme machine learning method and a modified multi-objective genetic algorithm called NSGA-II-aJG are adopted for classification and feature reduction, respectively. Finally, the proposed approach was implemented on the CICIDS2017 dataset and achieved an accuracy of 99.90%.

A feature selection algorithm based on the pigeon-inspired optimizer algorithm is presented as an intrusion detection system in Ref. [35]. This paper uses an s-shape

method and cosine similarity to turn the pigeon-inspired optimizer algorithm into a binary one. Finally, the proposed approach is implemented on three datasets NSL-KDD, UNSW-NB15, and KDDCUP99. The results of various experiments proved that the proposed method performed better regarding TPR, FPR, accuracy, and F1 criteria than several feature selection algorithms.

In Ref. [20], a new hybrid meta-heuristic method based on the fruit flight optimization algorithm and the ant lion optimization algorithm is presented to select and identify key features in the basic intrusion detection data set. Finally, the proposed method is implemented on three main intrusion detection datasets, including KDD Cup99, NSL-KDD, and UNSW-NB15, and has achieved 99.7%, 99.3%, and 99.1% accuracy, respectively, which is better compared to the other approaches.

In Ref. [36], a convolutional neural network method for feature extraction and an improved version of the transient search optimization algorithm called TSODE are developed. Finally, the proposed method is implemented on four main intrusion detection datasets, including KDDCup99, NSL-KDD, BoT-IoT, and CICIDS-2017 datasets. The results of various tests show that the proposed method has achieved better accuracy on most test data sets than other methods such as PSO, GWO, and TSO.

In Ref. [37], the PSO algorithm with inertia weight change is used for adaptive optimization of one-dimensional CNN structure parameters. The simulation results show that the APSO-CNN algorithm has effectively detected several types of network intrusion attacks.

In Ref. [38], a new IoT dataset called TON_IoT is generated and evaluated using different machine learning models. This research tests methods such as Deep Neural Networks (DNN) and GBM on the generated dataset. The results show that the DNN and GBM methods have achieved almost the same accuracy of 93.83%. An intrusion detection method based on the TON_IoT dataset in the VANET network was presented in Ref.[39]. The Chi-square technique (Chi 2) is used to select the feature, and the oversampling technique is used to balance the classes. The results of the tests on the ToN_IoT data set show that the XGBoost method, with an accuracy of 99.91%, has performed better than other methods. In Ref. [40], a combination of machine learning methods such as decision tree, naive Bayes, and random forest are applied in the first level. XGBoost uses the resulting classification results to identify normal and attack samples. Various results show that the proposed method on the TON-IoT dataset can achieve a recognition rate of 99.98% and an accuracy of 96.35%.

Literature on new intrusion and anomaly detection methods in the IoT was reviewed. Methods our review shows that machine learning and deep learning methods have received more attention in recent years. In these, the speed or time to detect attacks is very important. Usually, in intrusion detection systems, there are big data and high dimensions, which lead to the creation of models with complex calculations. Therefore, reducing or selecting effective features and extracting new features from the data set can play an effective role in increasing the speed and accuracy of intrusion detection systems. Optimization algorithms such as the GTO can effectively remove redundant features and select the main features of a dataset. Also, using deep learning algorithms helps extract new features. This paper presents a multi-objective enhanced GTO for feature selection to solve these problems and challenges. Also, a convolution neural network with hybrid layers called IOTFECNN has been designed and implemented to extract features to better detect anomalies in the IoT.

## 3 Gorilla Troops Optimizer

The GTO algorithm is one of the new meta-heuristic optimization algorithms introduced in 2021 [27]. The way in which gorillas naturally forage in the forest serves as the model for this algorithm. This algorithm creates new mechanisms to perform exploration and exploitation while also mathematically formulating the collective life of gorillas. A silverback leader, who oversees all of the group's decisions, leads them as they continue to live together and hunt for food as a group. Given GTO's special characteristics in many optimization problems and the fundamental ideas of Gorilla group life when searching for food and their collective existence, the algorithms can be widely applied [27].

### 3.1 Exploration Phase

For the exploration phase, three different mechanisms were used: migration to an unknown location, migration to a known location, and movement to other gorillas. Each of these three mechanisms is chosen using a standard procedure. A silverback gorilla is the best candidate solution at each optimization operation stage. The three mechanisms

used in the exploration phase were simulated using Eq. (1) [27].

$$GX(t+1) = \tag{1}$$

*GX (t + 1)* is the gorilla candidate position vector in the next iteration. *X(t)* is the current vector of the gorilla position. *p* is a parameter that must be given a value before the optimization operation; This parameter controls the likelihood of selecting a migration mechanism to an unknown location. $X_r$ is one member of the gorillas in the group randomly selected from the entire population and also *GXr.* One of the vectors of gorilla candidate positions was chosen at random and includes the positions that were updated in each phase. Finally, using Eqs. (2) and (3), *B* and *L* are calculated [27].

$$B = F \times \left(1 - \frac{}{MaxIT}\right) \tag{2}$$

$$L = B \times \text{l}. \tag{3}$$

In Eq. (4), It is the current iteration value, *MaxIt* is the total value of iterations to perform the optimization operation. *L* is calculated using Eq. (3) to simulate the silverback leadership.

The cost of all *GX* solutions is calculated at the end of the exploration phase, and if the cost is $GX(\text{t}) < X(\text{t})$, the *GX (t)* solution is used as the *X (t)* solution. As a result, the best solution generated during this phase is also regarded as a silverback [27].

### 3.2 Exploitation Phase

Two behaviors—follow the silverback and compete for adult females—are used in the exploitation phase of the GTO algorithm. If $C \geq W$, equation determines which version of the follow-the-silverback mechanism to use Eq. (4).

$$GX(t+1) = L \times M \times (X(\text{t}) - \text{Xsilverback}) + X(\text{t}), \tag{4}$$

$$M = \left(\left|\frac{1}{N}\sum_{i=1}^{n} GXi(t)\right|^{g}\right)^{1/g}. \tag{5}$$

In Eq. (4), *X(t)* is the gorilla position vector, and X*silverback* is the silverback gorilla position vector (best solution). In Eq. (5), *GXi(t)* shows each candidate gorilla's vector position in iteration t. if $C < W$, adult females' Competition according to Eq. (6) is taken.

$$GX(\text{i}) = \text{Xsilverback} - (\text{Xsilverback} \times Q - X(\text{t}) \times Q) \times A, \tag{6}$$

$$Q = 2 \times r5 - 1. \tag{7}$$

Equation is used to calculate the impact force, and *Q* is used to simulate it Eq. (7).

A group formation operation is carried out at the conclusion of the exploitation phase, during which the costs of all *GX* solutions are estimated. If the costs of $GX(t) < X(t)$ are equal, the *GX (t)* solution is used as the *X (t)* solution, and the best solution discovered among the entire population is viewed as a silverback [27].

## 4 The Proposed Method

In this section, an anomaly detection system based on the convolutional neural network and improved multi-objective gorilla troops optimizer is designed to detect and classify anomalies based on the type of attack in the IoT. Based on this, a convolutional neural network with hybrid layers called feature extraction convolution Neural Network IoT (FECNNIoT) is designed and implemented to extract features to better detect anomalies in the IoT. Moreover, an improved multi-objective binary version of the Gorilla algorithm called binary multi-objective enhanced Gorilla troops optimizer (BMEGTO) is introduced to select effective features. Finally, by combining the FECNNIoT and BMEGTO methods, a new hybrid method called CNN-BMEGTO-KNN is introduced. The proposed method CNN-BMEGTO-KNN is implemented and tested on NSL-KDD and TON-IoT datasets. In the proposed method, the BMEGTO algorithm is used to select the effective features in intrusion detection so that the selection of 20–50% of the features leads to an increase in the speed and accuracy of the proposed method.

On the other hand, the convolutional neural network FECNNIoT increases the accuracy and quality of the features by extracting separate features and combining them with the features selected by the BMEGTO algorithm. The general structure of the proposed method for anomaly detection on the IoT is reflected in Fig. 1. The most important parts of the proposed method are dataset, pre-processing, feature extraction based on convolutional neural network FECNNIoT, feature selection with binary multi-objective enhanced gorilla troops optimizer BMEGTO, Using the KNN Classifier method as an anomaly detector in the IoT. Each of the five parts is described in detail below.

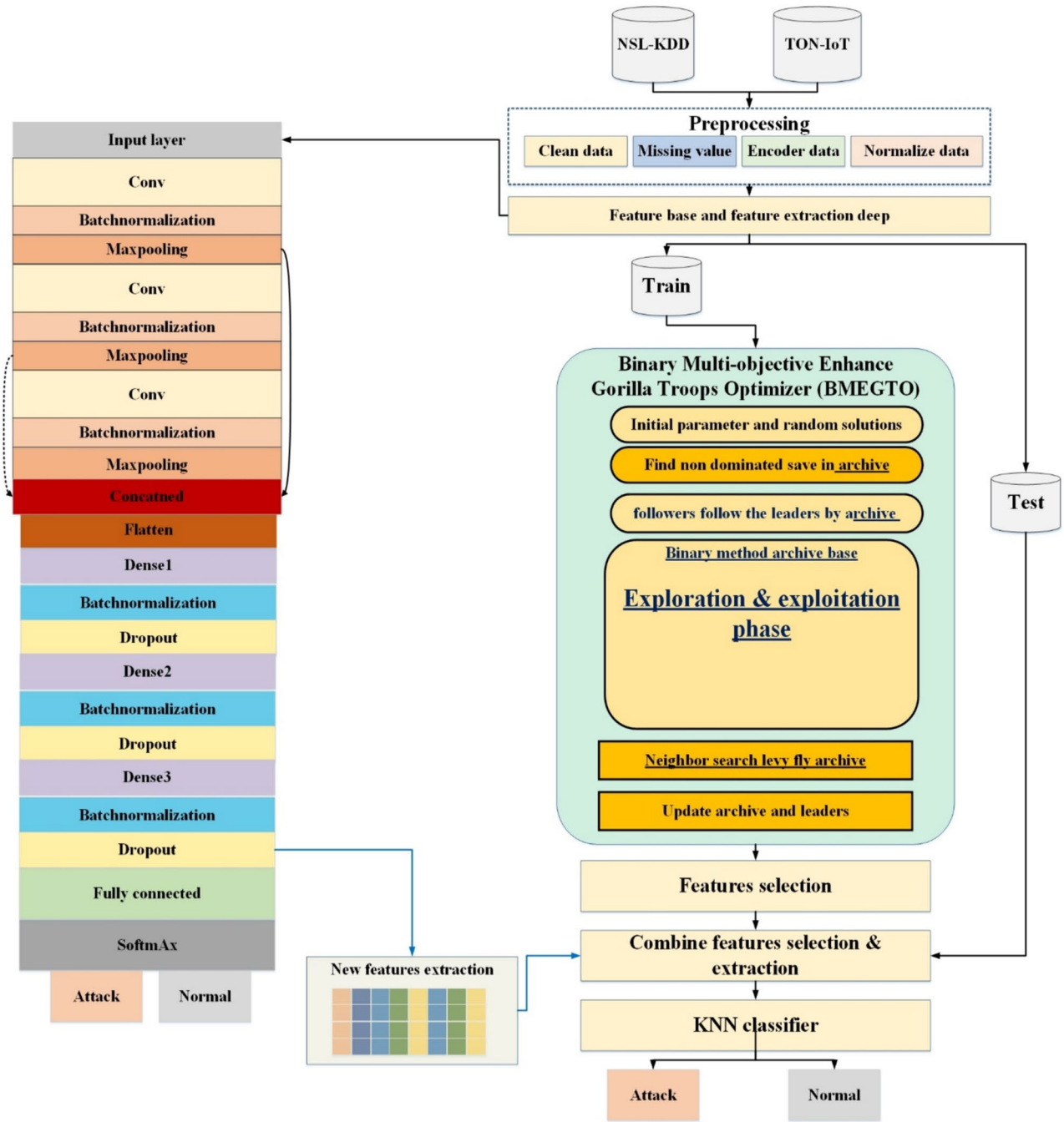



**Fig. 1** The general structure of the proposed method for anomaly detection in IoT

**Table 2** NSL-KDD as a two-class data set

| IoT traffic | Training samples | Testing samples |
|---|---|---|
| Normal | 67,343 | 9711 |
| Attack | 58,630 | 12,833 |

**Table 3** TON-IoT as a two-class dataset

| IoT traffic | Training& testing samples | Rows records |
|---|---|---|
| Normal | 300,000 | 796,380 |
| Attack | 161,043 | 21,542,641 |

### 4.1 Data Collection

NSL-KDD and TON-IoT are two datasets used to evaluate the proposed method. Each data set has different samples, and characteristics are described below in the form of two separate classes.

The NSL-KDD dataset contains 148,517 samples in two training and test parts named $KDD_{Train+}$ and $KDD_{Test+}$ [41], which have 125,973 and 22,544 samples, respectively. The NSL-KDD dataset with 41 features in two separate classes, including normal and abnormal samples, is reflected in Table 2.

The TON_IoT dataset is a new generation of IoT/IIoT datasets, network traffic, and operating systems [42]. This dataset has 22,390,223 records, including normal and attack data, of which 461,043 records are extracted from the Internet of Things traffic (300,000 records from normal traffic and 161,043 records from attack traffic). This data set with 44 features and 8 attack types, including normal and abnormal samples, is reflected in Table 3.

### 4.2 Preprocessing

This section introduces the preprocessing techniques applied to two datasets, NSL-KDD and TON-IoT. The first step of preprocessing is data cleaning or removing noisy data after examining the samples in the dataset. The second step of preprocessing techniques is filling missing values with suitable ones. Numeric features are filled using average values and string or batch data based on the most frequency method. The third step is data conversion turning the string or batch features into the corresponding numeric type with the label-encoder method. The fourth step is data normalization. At this step, all the features are placed in the range [0, 1] with the Min–Max normalization method defined as Eq. (8):

$$X_{norm} = \frac{X - X_{min}}{X_{max} - X_{min}}. \tag{8}$$

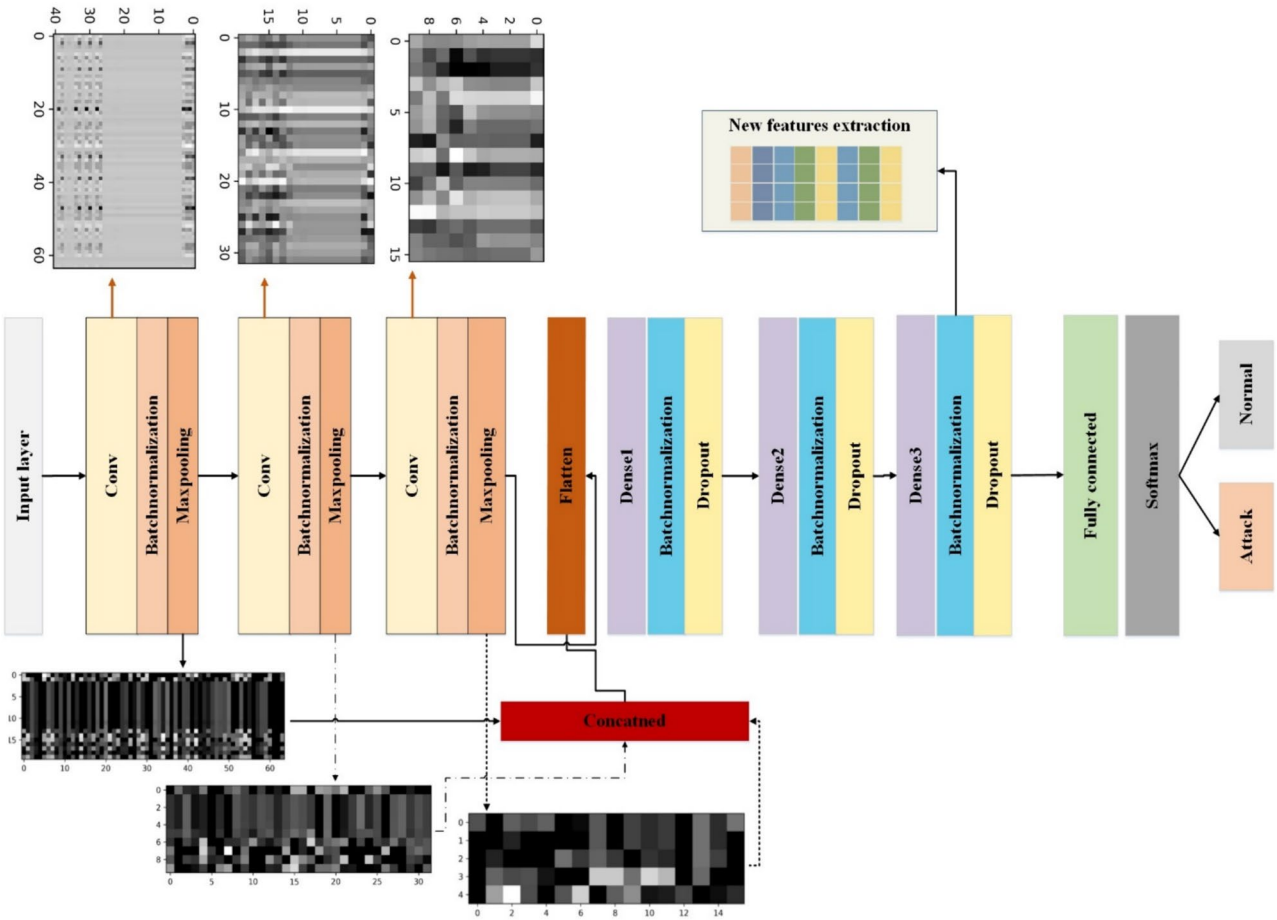



**Fig. 2** Structure of the proposed FECNNIoT architecture

**Fig. 3** Details of proposed neural network layers FECNNIoT

```
Index 00 , layer nanme input_features and shape :( None,41, 1)
index 01 , layer nanme Conv1D_1 and shape :( None,41, 64)
index 02 , layer nanme BatchNormalization_conv1 and shape :( None,41, 64)
index 03 , layer nanme Activation_conv1 and shape :( None, 41, 64)
index 04 , layer nanme MaxPool1D_conv1 and shape :(None,20, 64)
index 05 , layer nanme Conv1D_2 and shape :(None,20, 32)
index 06 , layer nanme BatchNormalization_conv2 and shape :(None, 20, 32)
index 07 , layer nanme Activation_conv2 and shape :(None,20, 32)
index 08 , layer nanme MaxPool1D_conv2 and shape :(None,10, 32)
index 09 , layer nanme Conv3 and shape :(None,10, 16)
index 10 , layer nanme BatchNormalization_conv3 and shape :(None,10, 16)
index 11 , layer nanme Activation_conv3 and shape :(None,10, 16)
index 12 , layer nanme MaxPool1D_conv3 and shape :(None,5, 16)
index 13 , layer nanme flatten_15 and shape :(None,1280)
index 14 , layer nanme flatten_16 and shape :(None,320)
index 15 , layer nanme flatten_17 and shape :(None,80)
index 16 , layer nanme Concatenate_layer and shape :(None,1680)
index 17 , layer nanme Dense_1 and shape :(None,512)
index 18 , layer nanme BatchNormalization_Dense1 and shape :(None,512)
index 19 , layer nanme Activation_Dense1 and shape :(None,512)
index 20 , layer nanme Dropout_Dense1 and shape :(None,512)
index 21 , layer nanme Dense_2 and shape :(None,256)
index 22 , layer nanme BatchNormalization_Dense2 and shape :(None,256)
index 23 , layer nanme Activation_Dense2 and shape :(None,256)
index 24 , layer nanme Dense_3 and shape :(None,10)
index 25 , layer nanme out_layer and shape :(2)
```

In Eq. (16), $X_{min}$ is the minimum value of each feature, $X$ is the value of each feature, and $X_{max}$ indicates the maximum value of each feature. Once the preprocessing steps are completed, both datasets are ready for machine learning and deep learning methods.

### 4.3 Proposed Convolutional Neural Network

A convolutional neural network is a special type of multilayer perceptron neural network. This neural network usually includes an input layer, an output layer, and hidden layers of convolutional layers. Also, convolutional layers with several filters of different dimensions are followed by a pooling layer [43, 44]. 2D and 3D convolutional neural networks are mostly used in image processing. But when faced with one-dimensional data, a one-dimensional convolutional neural network can play an important role in extracting new features from them. In this section, an advanced one-dimensional convolutional neural network called FECNNIoT is designed to extract low and high-level features to increase anomaly detection accuracy on the IoT. Figure 2 shows the proposed method's architecture and structure.

As shown in the FECNNIoT architecture, this network consists of three one-dimensional convolutional layers ([conv1, con2, conv3]), three one-dimensional pooling layers ([MaxPool1D_conv1, MaxPool1D_con2, MaxPool1D_conv3]), and three dense layers ([Dense1, Dense2, Dense3]). In addition, Dropout and batch normalization layers are used to increase efficiency and prevent over-fitting. More details on size, filters, and activation functions are provided below. The details of the neural network layers of the proposed method are reflected in Fig. 3.

According to Fig. 3, this network uses three convolutional layers along with batch normalization and pooling layers to extract low and high-level features. To extract low-level features, features extracted from different pooling layers [MaxPool1D_conv1, MaxPool1D_con2, MaxPool1D_conv3] are combined. The outputs number of MaxPool1D_conv1, MaxPool1D_conv2 and MaxPool1D_conv3 are equal to (20, 64), (10, 32) and (5, 16), respectively. Also, the first, second, and third pooling layers extract 1280, 320, and 80 feature maps, respectively. The concatenate layer compiles all these extracted features, and 1680 low-level feature maps are obtained. These 1680 low-level feature maps are used as input to the final dense layers to extract high-level features. Each of the last dense layers, whose number of neurons equals 512, 256, and 10, can be considered features extracted from the proposed neural network. This research extracts the Dense_3 layer with 10 features from the proposed neural network. Finally, these 10 features are combined with the features selected by the proposed algorithm BMEGTO and are considered as the input of classification algorithms. Therefore, there is no concern about the lack of access to basic features because the basic features are selected separately by the proposed algorithm BMEGTO.

Dropout and batch normalization layers are used in the proposed method to solve the over-fitting challenge in the FECNNIoT method. Dropout layers enhance neurons so that they learn features independently and are less dependent on other neurons. This process ultimately leads to an increase in the learning rate. In the FECNNIoT method, a Dropout layer called Dropout_Dense1 is set to 0.2. Batch normalization provides a higher learning rate by reducing internal variations and is used in most hidden layers. The last layer of the proposed method is designed as a Softmax layer to classify all types of attacks into two classes (normal and abnormal). The Softmax layer acts as a classifier of the FECNNIoT architecture. The Softmax layer is a generalization of logistic regression to the multi-class classification problem. The Softmax function is defined in Eq. (9).

$$f_i = \sum\nolimits_j h_i w_{ji}, \tag{9}$$

$$\sigma\left(\vec{f}_i\right) = p_i = \frac{e^{f_i}}{\sum_{j=1}^{C} e^{f_j}} label_{final} = \text{ argmax } \left(p_i\right), \tag{10}$$

where $C$ is the number of classes, $\sigma$ is the symbol of the Softmax function, $\vec{f}_i$ is the input vector, $e^{f_i}$ represents the standard exponential function for the output vector. Also, $h_i$ is the activation function in the penultimate layer and $w_{ji}$ is a weight number that communicates between the penultimate and softmax layers. Finally, the class label obtained from the Argmax function is considered the final output of the network for each input sample.

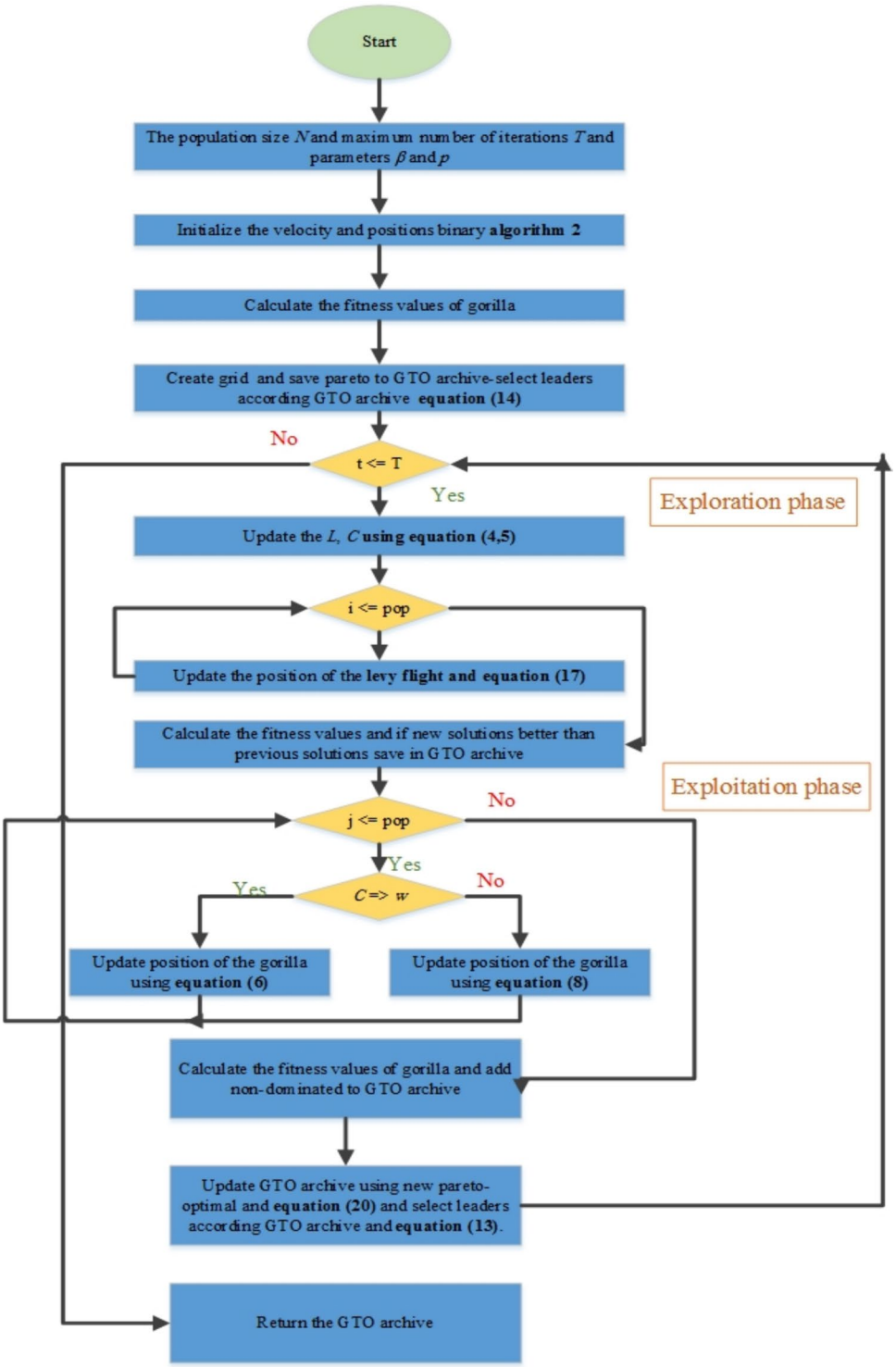
### 4.4 Binary Multi-objective Enhanced Gorilla Search Algorithm

In this section, a binary multi-objective enhanced gorilla troops optimizer called BMEGTO is developed, and its operators are designed in binary form to solve the feature selection problem. The changes applied to the proposed algorithm BMEGTO include the following: (1) use of archives to store the Pareto solution, (2) use of binary and genetic operators to provide the binary version, (3) using Levy flight to improve Gorilla search performance and increasing the exploration space (4) using the elite population inside the archive for multi-objective mode and increasing the efficiency of the BMEGTO algorithm (5) using grading and roulette wheel selection to select from the archive and the optimal solution.

For better understanding, the BMEGTO flowchart is shown in Fig. 4. The GTO algorithm uses various mechanisms for optimization operations, which are shown below.

The steps of the BMEGTO algorithm are explained step by step below. The pseudo-code of the binary multi-objective enhanced Gorilla troops optimizer is reflected in Algorithm 1.

**Fig. 4** Flowchart of BMEGTO



**Algorithm 1** Pseudo code of the BMEGTO

```
% GTO setting
Inputs: The population size N and maximum number of iterations T and parameters β and p
Outputs: The location of Gorilla and its fitness value
% initialization
Initialize the random population Xi (i = 1, 2, …, N)
Initialize the velocity and positions binary Algorithm 2.
Calculate the fitness values of Gorilla
Create Grid and Grid Indices and save pareto to GTOArchive
Select leaders according GTOArchive and Eq. (12).
% Main Loop
while (stopping condition is not met) do
Update the C using Eq. (2)
Update the L using Eq. (3)
% Exploration phase
for (each Gorilla (Xi)) do
        if (rand < p) then
        Update the position of the Levy flight and Eq. (15)
        End if
        if (rand ≥ 0.5) then
        Update the location Gorilla using Eq. (1)
        End if
        if (rand < 0.5) then
        Update the location Gorilla using Eq. (1)
        End if
end for
% Create group
Calculate the fitness values of Gorilla
if GX is better than X, replace them
Set Xsilverback as the location of silverback (best location)
% Exploitation phase
for (each Gorilla (Xi)) do
        if (|B| ≥ 1) then
        Update the location Gorilla using Eq. (6)
        Else
        Update the location Gorilla using Eq. (8)
        End if
end for
% Create group
Calculate the fitness values of Gorilla
 Add Non-Dominated to GTOArchive
 update GTOArchive using new pareto-optimal and Eq. (18).
 select leaders according GTOArchive and Eq. (11).
 Update Grid and Grid Indices
end while
Return XBestGorilla, bestFitness
```

#### 4.4.1 Binary Population Generation

The first step in the BMEGTO algorithm is to generate a random population. We are looking to generate binary solutions to the feature selection problem, so we need an initial binary population. To do this, for each dimension of the problem, if the value of *rand* is greater than 0.5, the value of 1 is assigned to it; otherwise, it takes the value of 0. How to generate a binary population is reflected in Algorithm 2.

**Algorithm 2** Generate binary population in the BMEGTO

**Binary Random initial**
**01: N is number of population**
**02: D is number of dimensions**
**03: For i= 1: N**
04: **For** j=1 to dim
05: *r*=rand()
04: **IF** ( *r*> 0.5 )
06: $X_i(j) = 1$
09: **Else**
10: $X_i(j) = 0$
11: **End if**
12: **End for j**
**13:** $fit_i = fitness(X_i)$ , $Classification_{error}$
12: **End for i**

Algorithm 2 leads to the random generation of binary numbers to create a population in the feature selection problem, where i refers to the ith member of the population and *j* refers to the jth dimension of the *i*th solution, and the rand function generates continuous numbers between 0 and 1. After generating random solutions, the values of the multi-objective function, including the number of selected features $N_{features}$ and the classification error rate $Classification_{error}$ are stored in $fit_i$ for the next steps.

#### 4.4.2 Storing in the Archive

The first step in multi-objective optimization [45] is to create an archive to store Pareto optimal solutions. In the proposed algorithm BMEGTO, an archive named GTOArchive is designed to store Pareto optimal solutions. In GTOArchive, the objective function space is represented in the form of grading, and the non-dominated solutions are included with the number of features and the classifier's error rate.

#### 4.4.3 Updating Velocity with Elitism

The second step in the BMEGTO algorithm is to update SilverBack across all population solutions. In the basic algorithm, this step is performed by Eq. (6), in which the objective function is a single-objective one, and the best solution Xsilverback is easily identifiable. But when there are several distinct objectives, selecting the best solution, i.e. a non-dominated solution, is extremely challenging. To do this, the best solution must be selected from the non-dominated solutions stored in GTOArchive. The selection operation is usually done randomly using the roulette wheel technique or based on grading.

In Ref. [29], and Ref. [46], the significance of feature selection and classification error is considered equal to 0.01 and 0.99, respectively. We changed the elite selection and grading based on the roulette wheel method so that more accurate leaders are more likely to be selected. Therefore, after grading the GTOArchive space, the roulette wheel mechanism with probability (*Pi*) for selecting each zone can be defined as follows:

$$P_i = \frac{e^{-\beta*(n_i+\varnothing)}}{\sum_{i=1}^{N} e^{-\beta*(n_i+\varnothing)}}. \tag{11}$$

In Eq. (11) $n_i$ is equal to the number of Pareto solutions obtained in *i*th zone. Consequently, $\varnothing$ assigns more probability of selection to zones with less classification error. In this paper, the parameter $\varnothing$ is set to 2. The higher the parameter $\beta$, the better the result. In this paper, the value of $\beta$ is set to 2. After selecting the grade, one of the solutions is selected as the leader, and this process is applied to all population members in the form of an Eq. (12).

$$GX(t+1) = L \times M \times (X(t) - \text{Leader}_k) + X(t). \tag{12}$$

Therefore, the velocity of all solutions in the proposed algorithm is updated via Eq. (12), in which *Leaderk* selected from GTOArchive is one of Pareto's optimal solutions.

#### 4.4.4 Improved Local Search Based on Levy Flight

The search space uses Levy flight-in meta-heuristic algorithms to create more diversity. This process causes significant changes in the search space for solutions. Levy flight creates better and longer steps by generating random solutions and moves the Gorilla algorithm away from local minima. In the normal motion pattern of Levy flight, the particle moves locally with several small steps, then takes a large step and moves locally again. From a mathematical

point of view, the Levy flight probability density function is defined as follows:

$$Distribution\ L\acute{e}U/|V|^{1/\beta}, \tag{13}$$

where β represents the power-law index, $V$ represents a random number sampled from the Gaussian distribution $N(0,1)$, and $U$ represents a random number sampled from the Gaussian distribution $N\left(0,\sigma^2\right)$. The following Equation expresses the standard deviation $\sigma$:

$$\sigma = \left( \frac{\Gamma(1+\beta) \times sin\left(\frac{\pi * \beta}{2}\right)}{\Gamma\left(\frac{1+\beta}{2}\right) \times \beta \times 2^{\frac{\beta-1}{2}}} \right)^{\frac{1}{\beta}}. \tag{14}$$

In the proposed algorithm, Levy distribution is used to improve exploration and better search. In the exploration phase, the GTO algorithm uses the *rand* function of Eq. (1) to migrate to an unknown location and searches the covered space based on the interval produced. This process results in better coverage of the search space. Therefore, in the BMEGTO, based on Levy flight is improved by Eq. (15).

$$GX(\mathrm{t}\ +1) = (UB - LB) \times Levy() + LB. \tag{15}$$

In fact, at this step, Levy flight guarantees to cover more search space. Equation (16) is used to turn this equation into a binary type.

$$x_j^i = \begin{cases} 1 \ \ if \ \ \delta\left(x_j^i\right) \geq rand \\ 0 otherwise \end{cases}, \tag{16}$$

$$\delta\left(x_j^i\right) = \frac{1}{1 + e^{10\left(x_j^i - 0.5\right)}}, \tag{17}$$

where *rand* produces a number between zero and one with a uniform distribution, and the function $\delta\left(x_j^i\right)$ is a sigmoid function mapping the solution to a new space between zero and one and finally leads to the transformation into binary equation.

#### 4.4.5 Archive Update

In the last step of the proposed method, the archive is updated based on new solutions. This phase is known as archive management in some algorithms. At this phase, the dominated solutions are not recorded in the archive. Also, if a new solution can overcome the Pareto optimal solutions in the archive, all prior dominated solutions are removed from the archive, and the new Pareto solution is recorded in the archive. If the archive is full and the number of non-dominated solutions exceeds the capacity of the archive, one of the solutions for objective function space with the largest population is randomly removed. This step is the reverse of the selection step. According to the mechanism of the roulette wheel, the probability of removal from densely populated areas is higher. This process reduces searching in unpromising populated areas. Therefore, Eq. (18) is used to remove areas based on the roulette wheel.

$$P_i\prime = \frac{n_i}{c}, \tag{18}$$

where $c$ is a positive constant number greater than 1 and $n_i$ is the number of Pareto optimal solutions obtained in the $i$th section.

### 4.5 KNN-based Proposed Objective Function

The k Nearest Neighbor (KNN) method is one of the most popular algorithm in clustering and data classification expanded by Thomas Cover [47]. This classifier does not undergo the training process and only stores the training data and uses it to classify the neighbors. It is used in most works because of its straightforward implementation and high performance.

In line with less feature selection and higher classification accuracy for anomaly detection, we define two objective functions for the BMEGTO algorithm. We are faced with two conflicting objectives in our feature selection problem. The first objective is to reduce the number of features and increase the speed of classification algorithms, and the second is to increase the classification accuracy. The two objective functions, including the number of features and the classification error rate, can be defined as Eq. (19).

$$MinimizeF(x) = \begin{cases} F_1(X) = N_{features} \\ F_2(X) = Classification_{error} \end{cases}, \tag{19}$$

where $F_1(X)$ is the first objective function for feature reduction, $F_2(X)$ is the second objective function representing classification error reduction. Also, $N_{features}$ and $Classification_{error}$ indicate the number of features and classification error, respectively. In this paper, we used the KNN classifier as the final classifier for anomaly detection on the Internet of Things due to its high detection speed and accuracy.

### 4.6 Evaluation Criteria

To evaluate the proposed approach CNN-BMEGTO-KNN and compare it with other algorithms, the main classification criteria, including accuracy, precision, recall, and F1, are used, the values of each of which vary between 0 and 1. Below each of the criteria is defined along with their formulas [44].

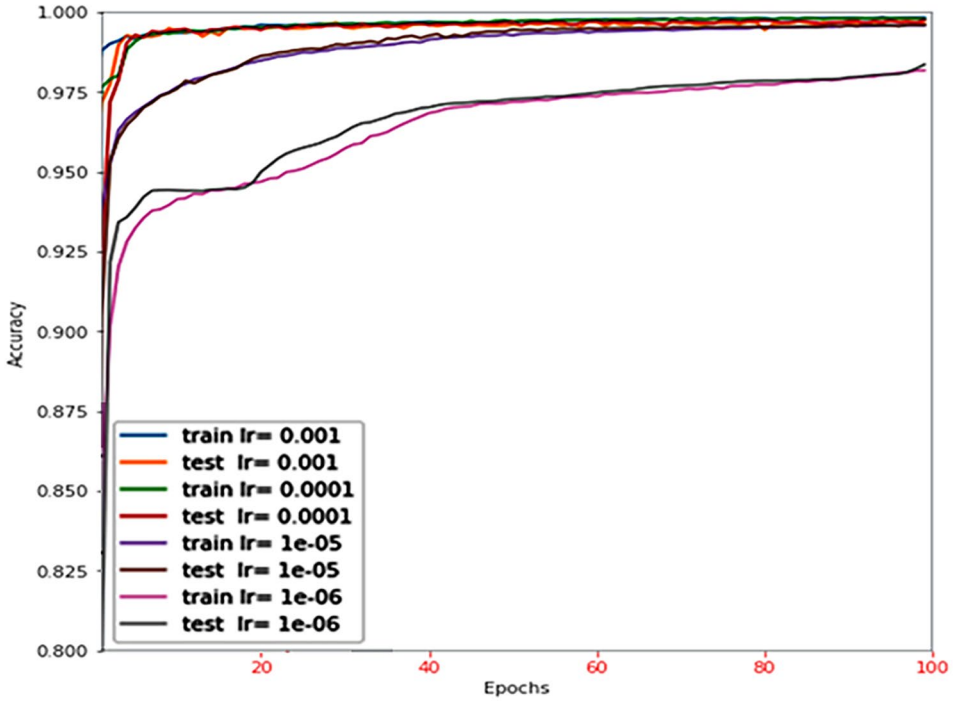



**Fig. 5** Setting the value of the learning rate parameter of the FECNNIoT

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}, \tag{20}$$

$$Precision = \frac{TP}{TP + FP}, \tag{21}$$

$$Recall = \frac{TP}{TP + FN}, \tag{22}$$

$$F1 = \frac{2 * Precision * Recal}{Precision + Recall}, \tag{23}$$

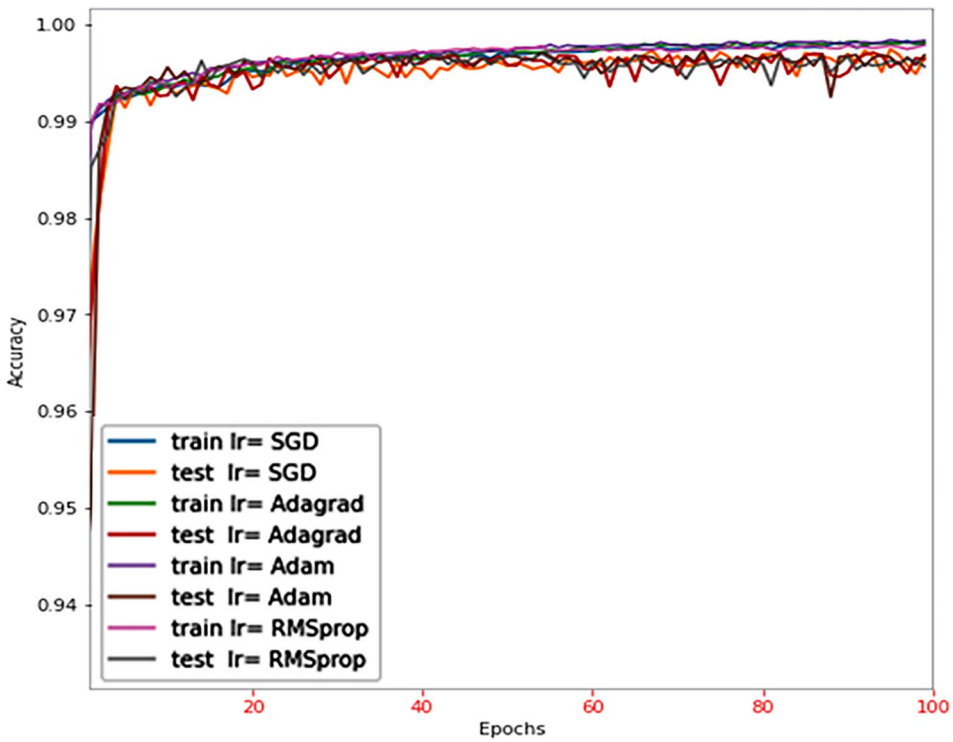



**Fig. 6** Setting the optimization parameter in the FECNNIoT

**Table 4** Selection of effective features using the BMEGTO method on the NSL-KDD dataset

| No | Number of features | Feature names | Accuracy |
|---|---|---|---|
| 1 | 1 | 'src_bytes' | 95.38 |
| 2 | 2 | 'src_bytes' "srv_count" | 95.94 |
| 3 | 4 | 'src_bytes' 'hot"srv_count"dst_host_srv_count' | 98.09 |
| 4 | 5 | 'src_bytes' 'srv_count"dst_host_srv_count"dst_host_serror_rate' 'dst_host_rerror_rate' | 98.72 |
| 5 | 7 | 'service"src_bytes' 'hot"srv_count"serror_rate' 'dst_host_same_srv_rate"dst_host_same_src_port_rate' | 99.38 |
| 6 | 9 | 'service"src_bytes' ' hot"num_shells"srv_count 'hot dst_host_same_srv_rate' 'dst_host_diff_srv_rate"dst_host_rerror_rate' | 99.44 |
| 7 | 11 | 'service"src_bytes' 'hot"num_shells"count"srv_count' 'serror_ rate"srv_diff_host_rate"dst_host_same_srv_rate' 'dst_host_diff_srv_rate"dst_host_rerror_rate' | 99.49 |
| 8 | 13 | 'duration"service"src_bytes' 'dst_bytes"su_attempted' 'is_host_login"srv_count"diff_srv_rate"dst_host_count' 'dst_host_same_src_port_rate"dst_host_srv_diff_host_rate' 'dst_host_serror_rate"dst_host_srv_serror_rate' | 99.54 |
| 9 | 16 | 'duration"protocol_type"service" 'src_bytes' 'dst_bytes"hot"num_ root"is_host_login"land" rerror_rate' 'diff_srv_rate"dst_host_count"dst_host_same_src_port_rate' "dst_host_serror_rate' 'dst_host_srv_serror_rate"dst_host_srv_rerror_rate' | 99.59 |
| 10 | 26 | 'duration"protocol_type"service"flag' 'src_bytes' 'dst_bytes"srv_count""land"su_attempted"num_root' 'num_access_files"is_host_login"count' 'srv_count"srv_serror_rate"srv_rerror_rate"same_srv_rate' 'diff_srv_rate"srv_diff_host_rate' 'dst_host_count"dst_host_srv_count"dst_host_diff_srv_rate' 'dst_host_same_src_port_rate' 'dst_host_srv_diff_host_rate"dst_host_serror_rate' 'dst_host_srv_serror_rate"dst_host_srv_rerror_rate' | 99.56 |

**Table 5** Comparison of the CNN-BMEGTO-KNN method with other methods on the NSL-KDD dataset

| Algorithm | Accuracy (%) | Precision (%) | Recall (%) | F1-score (%) |
|---|---|---|---|---|
| DT | 98.10 | 98.04 | 98.17 | 98.09 |
| KNN | 99.60 | 99.61 | 99.60 | 99.61 |
| MLP | 99.48 | 99.47 | 99.49 | 99.48 |
| NB | 89.81 | 90.43 | 89.39 | 89.65 |
| RF | 99.52 | 99.52 | 99.58 | 99.60 |
| LSTM | 99.55 | 99.55 | 99.55 | 99.55 |
| GRU | 99.33 | 99.34 | 99.31 | 99.32 |
| FECNNIoT | 99.65 | 99.66 | 99.65 | 99.65 |
| CNN-BMEGTO-KNN | **99.86** | **99.86** | **99.86** | **99.86** |

Bold numbers indicate the improvement of the proposed method

where in above equations, $N$ is the total number of samples in the dataset. $TP$, $FP$, $TN$, and $FN$ represent true positive, false positive, true negative, and false negative, respectively.

## 5 Results and Evaluation

In this section, two proposed methods are tested and evaluated, including the proposed hybrid method CNN-BMEGTO-KNN and the proposed deep network method FECNNIoT. The proposed approach CNN-BMEGTO-KNN

**Table 6** Comparison of the CNN-BMEGTO-KNN method with other methods on the NSL-KDD dataset regarding FPR and FNR criteria

| Algorithm | TP | TN | FN | FP | FPR | FNR |
|---|---|---|---|---|---|---|
| DT | 13,122 | 11,596 | 372 | 105 | 0.0090 | **0.0276** |
| KNN | 13,441 | 11,632 | 53 | 69 | 0.0059 | 0.0039 |
| MLP | 13,418 | 11,648 | 76 | 53 | 0.0045 | 0.0056 |
| NB | 12,868 | 9761 | 626 | 1940 | 0.1658 | 0.0464 |
| RF | 13,481 | 11,615 | 13 | 86 | 0.0073 | **0.001** |
| LSTM | 13,438 | 11,646 | 56 | 55 | 0.0047 | 0.0041 |
| GRU | 13,435 | 11,592 | 59 | 109 | 0.0093 | 0.0044 |
| FECNNIoT | 13,462 | 11,647 | 32 | 54 | 0.0046 | 0.0024 |
| CNN-BMEGTO-KNN | **13,480** | **11,679** | **12** | **25** | **0.0018** | **0.001** |

Bold values indicate the improvement of the proposed scheme

is divided into two distinct parts: feature selection and deep network design. MATLAB 2020 software implements the BMEGTO algorithm on a PC with a 5-core processor, 8 GB of RAM, and Windows 10 operating system. Also, the proposed deep method is implemented in TensorFlow and Keras frameworks. Data analysis and plotting are done using pandas, Matlabplot and Seaborn libraries. Classical Gorillae learning algorithms, including NB, DT KNN, MLP, and RF, are implemented with the Sklearn library. Deep learning algorithms, including CNN, LSTM [48], and GRU, are implemented in TensorFlow and Keras frameworks. In both NSL-KDD and TON-IOT datasets, 80% and 20% of the data are used for training and testing, respectively. In feature selection for the BMEGTO algorithm, 50% of the data are considered for training and 50% for testing. The parameters related to deep algorithms are set as follows: epochs equal to 100, batch size equal to 1000, learning rate equal to 1e-4. Adam's algorithm has also been used as an optimizer. Subsequently, the parameters of the machine learning methods in the Sklearn library are set as follows: the Max_depth parameter in DT equals 100, the Hidden_layer_sizes parameter in MLP equals [100,100], and the N_neighbors parameter in KNN equals 100.

### 5.1 Evaluating the Parameters of Deep Network FECNNIoT

In this section, important parameters affecting the output and accuracy of deep network FECNNIoT, such as learning rate, the first and most important parameter in any deep network, are tested on NSL-KDD and TON- IoT datasets. This section considers different learning rates for the deep network FECNNIoT, and the results are reflected in Fig. 5.

Based on the observations, the learning rate with the value of 1e–6 has resulted in the worst performance. Learning rates higher than 1e–6, including 1e–5 to 1e–1, have also been evaluated. The results of this experiment prove that the learning rate of 1e–4 gives the best performance. Below, we have evaluated the proposed method on four different optimizers, including "Adam," "SGD," "Adagrad," and "RMSprop," and the results are shown in Fig. 6. As shown in Fig. 6, "SGD" and "Adagrad" optimizers have more fluctuations on test data than other optimizers. The two optimizers, Adam and RMSprop, have less fluctuation on both training and test data and have obtained more accurate results. According to the various tests, the RMSprop optimizer was finally selected as the final optimizer in this research.

### 5.2 Evaluation on the NSL-KDD Dataset

The purpose of this section is to evaluate the effectiveness of the CNN-BMEGTO-KNN method on NSL-KDD datasets. The first test is related to the evaluation of BMEGTO in selecting effective features on the NSL-KDD dataset, i.e. the non-dominated solutions stored in the archive and examined in the last iteration of BMEGTO regarding the number of selected features and accuracy. The results of the feature selection process are shown in Table 4.

As shown in Table 4, the BMEGTO method with different sizes (the number of selected features with different accuracies) can effectively identify important features and remove redundant features. In general, we can choose any step as the final solution. For example, if we consider solution 7 with the number of 11 features and accuracy of 99.49% as the final solution, i.e., the BMEGTO method has been able to select about 27% of features as important features and remove 73% of features as redundant features. We can choose any solution in Table 5 according to different objectives as the final solution. The main objective of this study is to achieve the highest accuracy and lowest features. So we chose the 9th solution with 99.59 accuracy and 18 selected features as the final solution. Below, the CNN-BMEGTO-KNN method has been implemented with deep and machine learning algorithms on the NSL-KDD data set. Its results have been evaluated regarding precision, accuracy, and recall criteria. The obtained results are shown in Table 5.

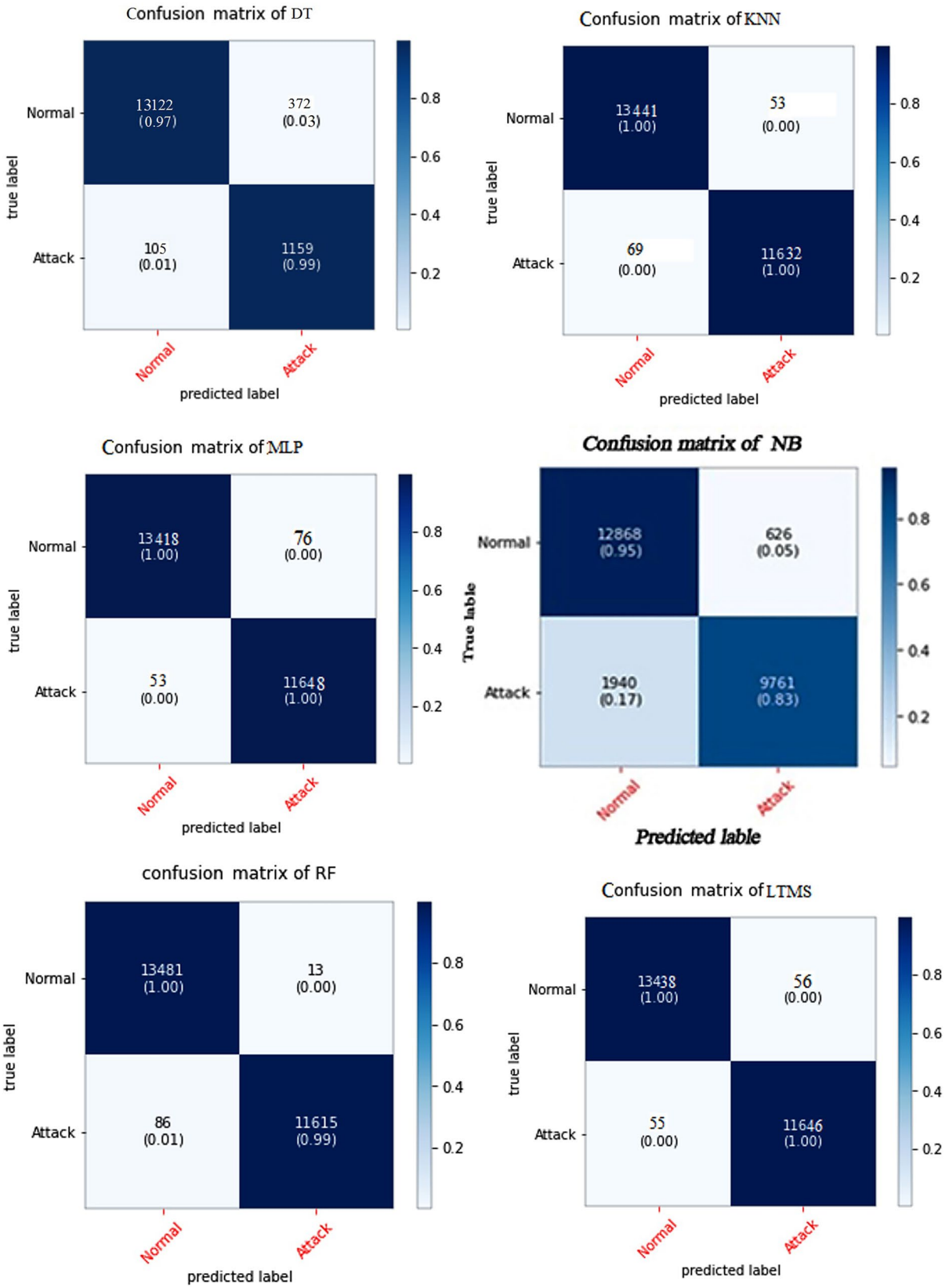



**Fig. 7** The results of the confusion matrix associated with each of the methods on the NSL-KDD dataset

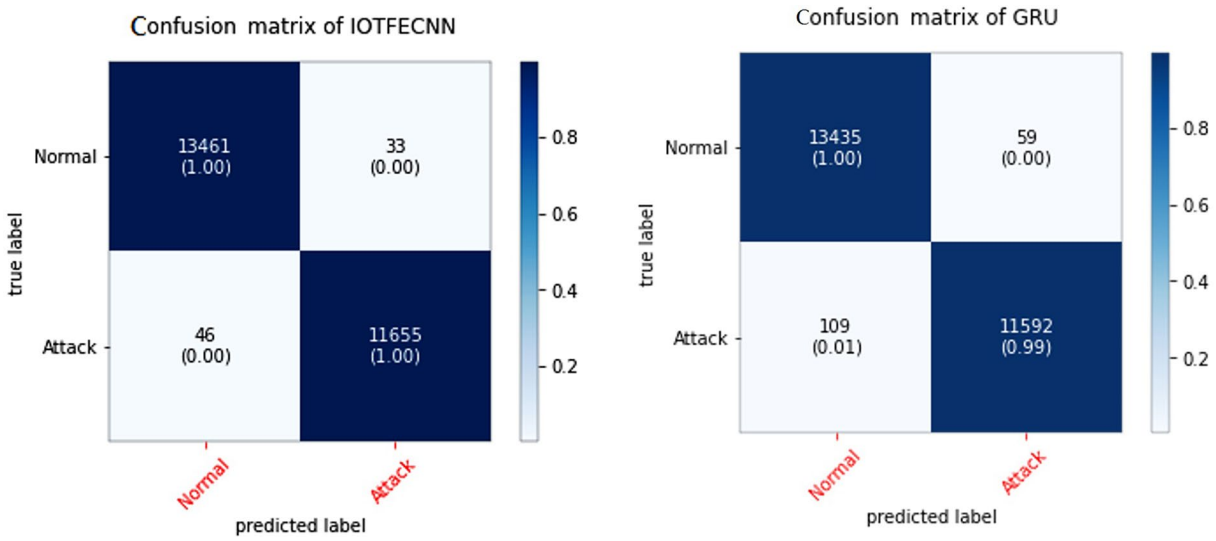



**Fig. 7** (continued)

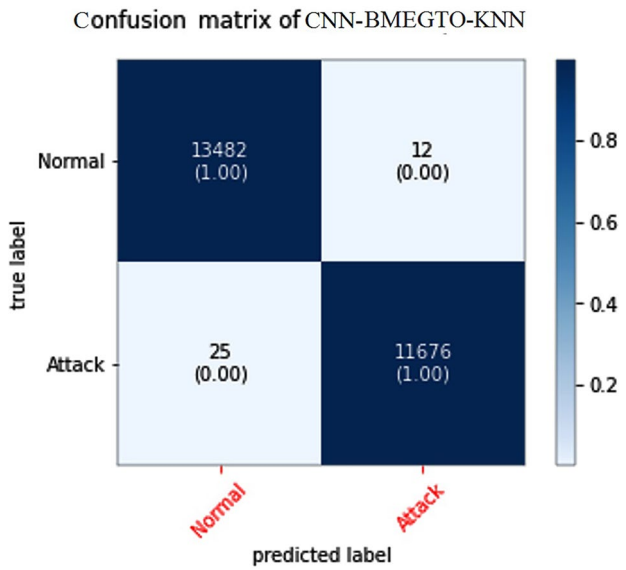



**Fig. 8** Confusion matrix of the CNN-BMEGTO-KNN method on the NSL-KDD dataset

A comparison of the results in Table 5 shows that the proposed method has recorded the best performance with 99.86 accuracies compared to deep and machine learning methods. In addition, the FECNNIoT method as a feature extraction method has achieved an accuracy of 99.65%. Deep learning methods such as LSTM and GRU have obtained comparatively weaker results with accuracies of 99.55% and 99.33%, respectively, compared to the proposed algorithm. Also, among machine learning methods, the KNN algorithm with 99.60% accuracy and the RF algorithm with 99.52% accuracy have shown better performance than other methods. The test results prove that the CNN-BMEGTO-KNN can be introduced as a reliable method in anomaly detection on the Internet of Things. For further evaluation, the proposed method regarding two other critical criteria, including false-negative rate (FNR) and False-Positive Rate (FPR), is compared, and the test results are presented in Table 6. Although both criteria are important, FNR can bring more risks to a system in attack detection.

Table 6 shows that the proposed method has achieved values of 0.0018 and 0.001 in FPR and FNR criteria, respectively, and has recorded better performance than other methods. To get enough insight into the number of true and false samples of attacks and the normal state of each of the methods in Table 6, the results of this table are plotted in the form of confusion matrices in Figs. 7 and 8.

The results of the confusion matrix of different methods in Fig. 7 show that the IOTFECNN method has wrongly recognized 32 and 54 samples in normal and attack classes, respectively, which is a better performance compared to deep and machine learning methods. However, the results of the confusion matrix of the CNN-BMEGTO-KNN method in Fig. 8 show that in the normal and attack classes, only 12 and 25 samples are wrongly recognized, respectively.

The comparison of the results of the proposed method with other references and new methods in terms of anomaly detection on the Internet of Things is shown in Table 7.

The comparison of the results shows that the proposed method has performed even better than the new method CNN-BMECapsa-RF in all four measures of Accuracy, Precision, Recall, and F1-score. Although the size of the change is very small, but it should be said that the number of test samples is more than 20 thousand and a slight change

**Table 7** Comparison of the CNN-BMEGTO-KNN method with other references and new methods

| Refs. | Algorithm | Accuracy (%) | Precision (%) | Recall (%) | F1-score (%) |
|---|---|---|---|---|---|
| [49] | Deep learning + SMO | 94.69 | – | – | – |
| [50] | Whale integrated LSTM (WILS) | 99.5 | – | 98.45 | – |
| [36] | CNN + TSO | 95.091 | 94.57 | 90.68 | 97.09 |
| | CNN + TSO + DE | 99.629 | 99.629 | 99.629 | 99.629 |
| [51] | Deep method | 99.20 | 99.19 | 99.21 | 99.20 |
| [28] | GWO–PSO–RF | 99.24 | – | – | – |
| [20] | FFA-ALO-SVM | 98.13 | – | 88.37 | – |
| | FFA-ALO-KNN | 99.06 | – | 89.56 | – |
| | FFA-ALO-NB | 75.10 | – | 82.70 | – |
| | FFA-ALO-DT | 99.33 | – | 99.24 | – |
| [35] | Sigmoid PIO | 86.9 | – | – | 86.4 |
| | Cosine PIO | 88.3 | – | – | 88.2 |
| [27] | NNIA + S1 subset | 99.38 | – | – | – |
| | NNIA + S2 subset | 99.47 | – | – | – |
| | NNIA + S3 subset | 99.44 | – | – | – |
| | NNIA + S4 subset | 99.39 | – | – | – |
| | NNIA + S5 subset | 99.46 | – | – | – |
| [32] | DNN-C | 98.84 | 99.15 | 98.36 | 98.68 |
| | DNN-20 | 98.74 | 99.32 | 97.96 | 98.63 |
| | DNN-KNN | 99.77 | 99.74 | 99.76 | 99.75 |
| [1] | RNN | 92.18 | 90.23 | – | 92.29 |
| [48] | RNN | 98.68 | – | – | 98.83 |
| | DNN | 98.95 | – | – | 99.07 |
| [49] | FFO-PNN | 98.99 | 96.97 | 96.97 | 96.97 |
| [50] | CNN-BMECapsa-RF | 99.85 | 99.85 | 99.85 | 99.85 |
| Proposed method | FECNNIoT | 99.73 | 99.72 | 99.72 | 99.72 |
| | CNN-BMEGTO-KNN | **99.86** | **99.86** | **99.86** | **99.86** |

Bold numbers indicate the improvement of the proposed method

**Table 8** Selecting the effective feature and increasing the accuracy with the BMEGTO method

| NO | No. features | Feature names | Accuracy |
|---|---|---|---|
| 1 | 8 | src_ip, src_port, src_bytes, ssl_version, http_method, http_status_code, http_resp_mime_types, dns_rejected | 99.998 |
| 2 | 9 | ts, src_ip, src_port, dst_ip, src_bytes, dst_ip_bytes, http_method, http_status_code, http_resp_mime_types | 99.934 |
| 3 | 11 | ts, src_ip, src_port, dst_ip, duration, src_bytes, dst_ip_bytes, dns_AA, dns_RD, http_status_code, http_resp_mime_types | 99.999 |
| 4 | 14 | ts, src_ip, dst_bytes, conn_state, src_pkts, dns_query, dns_AA, dns_RD, ssl_established, http_version, http_user_agent, http_orig_mime_types, http_resp_mime_types, weird_name | 99.997 |
| 5 | 16 | src_ip, src_port, dst_port, proto, src_bytes, dst_bytes, missed_bytes, src_pkts, dns_rejected, ssl_version, ssl_subject, ssl_issuer, http_method, http_version, http_request_body_len, weird_name | 99.9692 |
| 6 | 19 | ts, src_ip, dst_ip, duration, src_bytes, dst_bytes, missed_bytes, src_pkts, dst_pkts, dst_ip_bytes, ns_rcode, ssl_version, ssl_cipher, ssl_established, ssl_subject, http_response_body_len, http_orig_mime_types, weird_addl, dns_qtype | 99.998 |
| 7 | 25 | ts, src_ip, src_port, dst_ip, dst_port, proto, service, duration, src_bytes, src_pkts, dst_pkts, dst_ip_bytes, dns_query, dns_rcode, dns_AA, ssl_cipher, ssl_subject, http_trans_depth, http_method, http_uri, http_response_body_len, http_status_code, http_user_agent, http_resp_mime_types, weird_addl | 99.999 |
| 8 | 37 | ts, src_ip, src_port, dst_ip, dst_port, proto, duration, src_bytes, dst_bytes, conn_state, src_pkts, src_ip_bytes, dst_pkts, dst_ip_bytes, dns_query, dns_qclass, dns_qtype, dns_rcode, dns_AA, dns_RD, dns_RA, ssl_version, ssl_resumed, ssl_established, ssl_subject, ssl_issuer, http_method, http_uri, http_version, http_request_body_len, http_response_body_len, http_user_agent, http_orig_mime_types, http_resp_mime_types, weird_name, weird_addl, weird_notice | 99.998 |

**Table 9** Comparison of the CNN-BMEGTO-KNN method with other methods on the TON- IoT dataset

| Algorithm | Accuracy (%) | Precision (%) | Recall (%) | F1-score (%) |
|---|---|---|---|---|
| DT | 99.88 | 99.87 | 99.87 | 99.87 |
| KNN | 99.51 | 99.30 | 99.38 | 99.34 |
| MLP | 99.79 | 99.78 | 99.77 | 99.77 |
| NB | 88.56 | 87.17 | 89.98 | 87.97 |
| RF | 99.81 | 99.89 | 99.90 | 99.90 |
| LSTM | 99.79 | 99.77 | 99.77 | 99.77 |
| GRU | 99.79 | 99.80 | 99.75 | 99.77 |
| FECNNIoT | 99.83 | 99.81 | 99.82 | 99.81 |
| CNN-BMEGTO-KNN | **99.99** | **99.99** | **99.99** | **99.99** |

Bold numbers indicate the improvement of the proposed method

**Table 10** Comparison of the CNN-BMEGTO-KNN method with other methods on the TON- IoT dataset regarding FPR and FNR criteria

| Algorithm | TP | TN | FN | FP | FPR | FNR |
|---|---|---|---|---|---|---|
| DT | 59,864 | 32,239 | 56 | 50 | 0.0015 | 0.0009 |
| KNN | 59,503 | 31,985 | 417 | 304 | 0.0094 | 0.0070 |
| MLP | 59,830 | 32,194 | 90 | 95 | 0.0029 | 0.0015 |
| NB | 51,095 | 30,574 | 8825 | 1715 | 0.0531 | 0.1473 |
| RF | 59,870 | 32,257 | 50 | 32 | 0.0010 | 0.0008 |
| LSTM | 59,830 | 32,198 | 90 | 100 | 0.0031 | 0.0015 |
| GRU | 59,860 | 32,161 | 60 | 128 | 0.0040 | 0.0010 |
| FECNNIoT | 59,839 | 32,217 | 81 | 72 | 0.0022 | 0.0014 |
| CNN-BMEGTO-KNN | **59,919** | **32,258** | **0** | **2** | **0.0001** | **0.00002** |

Bold values indicate the improvement of the proposed scheme

in accuracy can play an important role in the number of true positive and negative samples.

This means that the performance of GTO algorithm with KNN classification is better than Capuchin algorithm and RF classification. In the tests and investigations, the basic GTO algorithm has performed better than the Capuchin basic algorithm in the Exploration Phase. This performance of the basic algorithm has made the algorithm more efficient in the combination of CNN-BMEGTO-KNN.

Also among the new methods, the DNN-KNN method has achieved an accuracy of 99.77 and has performed better than other methods. The proposed method CNN-BMEGTO-KNN with an accuracy of 99.86%, has a relative superiority compared to the DNN-KNN method. In addition, the NNIA method based on multi-objective feature selection has achieved an accuracy of 99.46% and has a weaker performance than the proposed method.

### 5.3 Evaluation on the TON-IoT Dataset

In this section, the proposed method is evaluated on the TON-IOT dataset and compared with other methods. The first experiment is related to the evaluation of BMEGTO in selecting effective features on the TON- IoT dataset, which uses non-dominated solutions stored in the multi-objective archive to check the number of selected features and the resulting accuracy.

The results show that the BMEGTO method can identify features in the TON- IoT dataset that led to increased accuracy. According to the objectives of the study, solution 4 in Table 8 with 12 features, i.e. about 25% of the features and with 99.999% accuracy, has been selected as the final solution. Below, the proposed method is compared with deep and machine learning algorithms on the TON-IoT data set in terms of criteria such as accuracy, precision, and recall. The results are presented in Table 9.

Table 9 compares the results regarding different criteria and shows that the CNN-BMEGTO-KNN method with 99.99% accuracy has achieved better performance than the deep and machine learning methods. The FECNNIoT method has achieved 99.83% accuracy without feature selection. Below, FPR and FNR criteria are evaluated in Table 10.

A comparison of the results in Table 10 shows that the FPR and FNR values of the proposed method are equal to 0.0001 and 0.00002, respectively. In addition, DT, RF, and FECNNIoT in the FPR criterion, i.e. relatively good

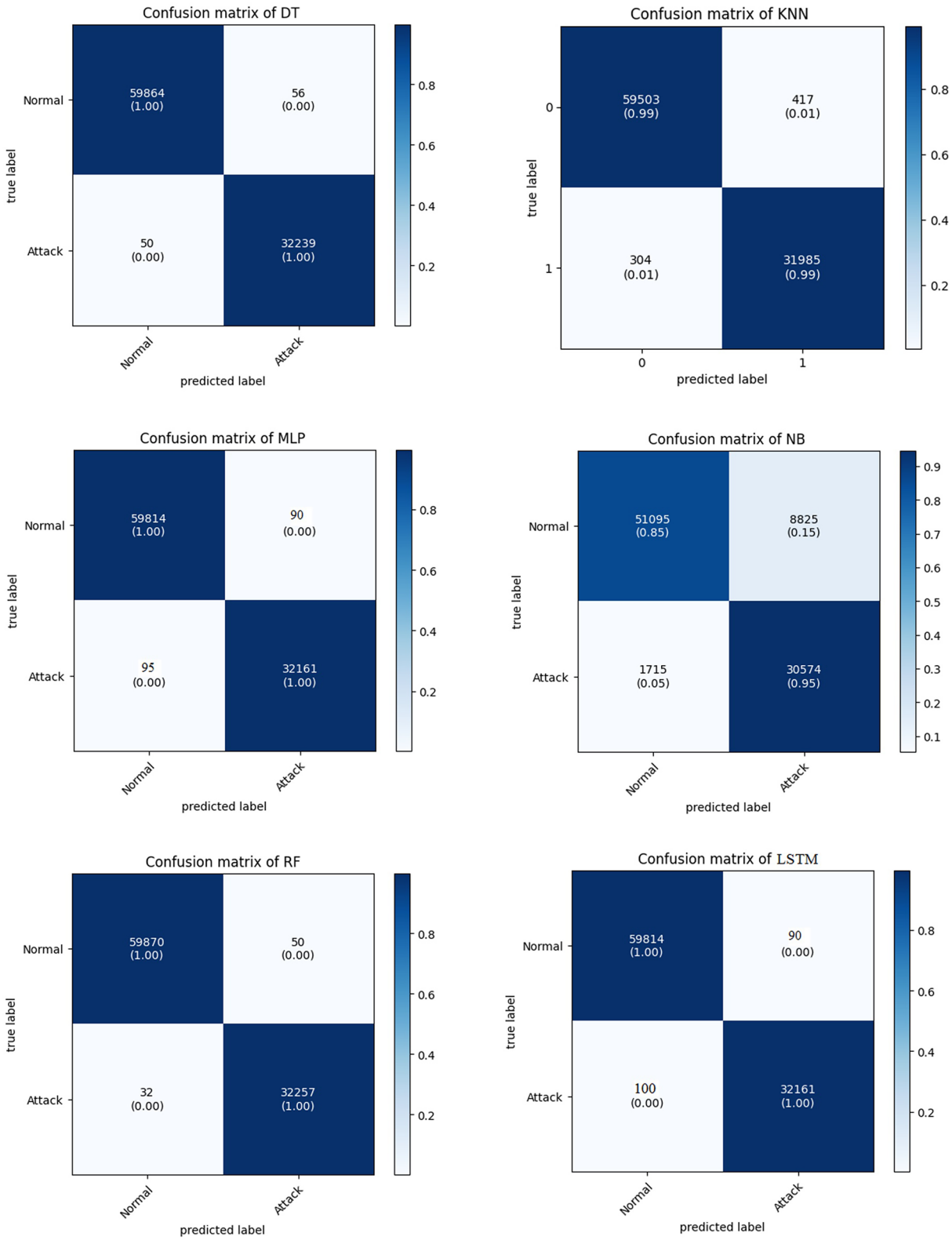



**Fig. 9** The results of the confusion matrix of each method on the TON-IOT data set

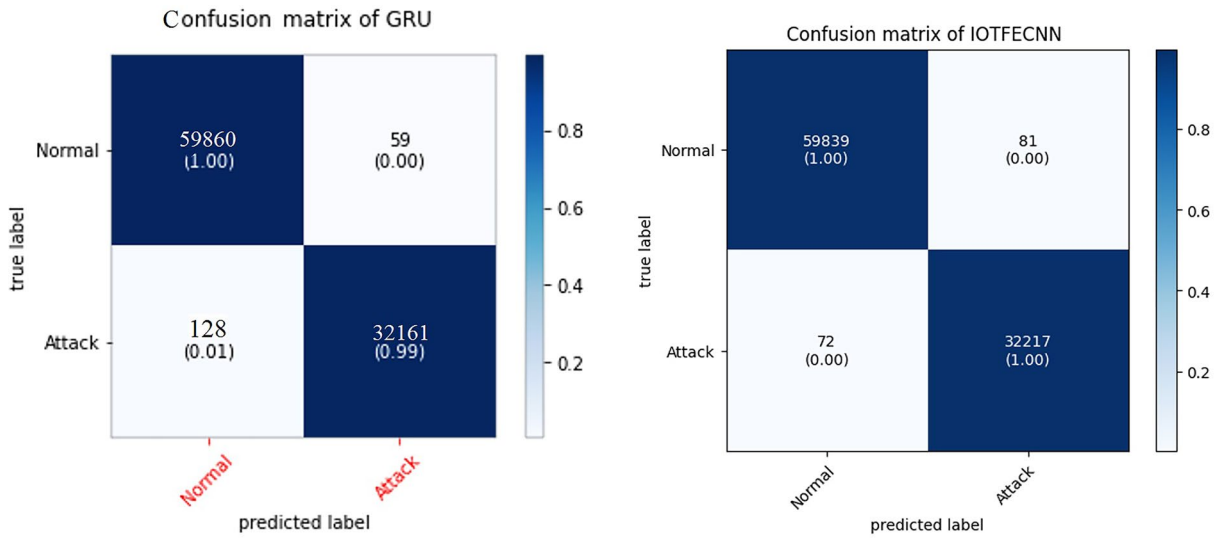



**Fig. 9** (continued)

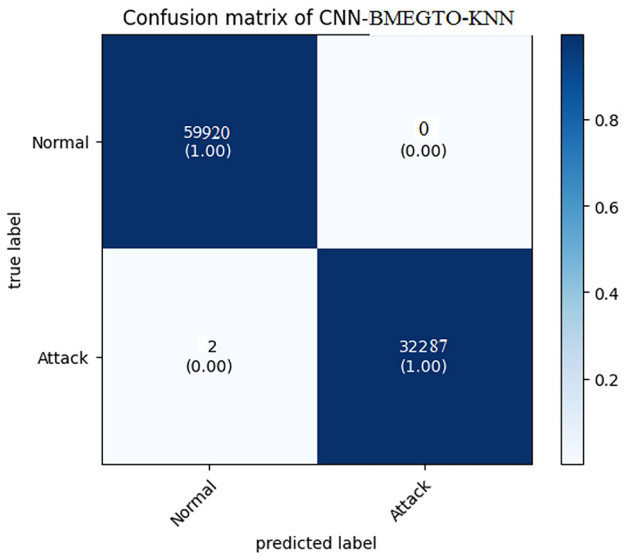



**Fig. 10** The results of the confusion matrix of the CNN-BMECapSA-RF method on the TON-IOT data set

performance compared to other methods. The results of this table are plotted in the form of confusion matrices in Figs. 9 and 10.

The results of each method show that the FECNNIoT method has detected 81 attack samples and 72 normal samples in the normal class and the attack class, respectively. The FECNNIoT method has performed poorly compared to some methods such as DT and RF but has performed better compared to other deep and machine learning methods. Also, the results of the CNN-BMEGTO-KNN method in Fig. 10 show that in the normal class and the attack class, 0 attack sample and 2 normal samples have been detected, respectively.

The results show that the CNN-BMEGTO-KNN method is powerful for detecting anomalies in the Internet of Things. Below, the results of the proposed method are compared with other references and new methods for detecting anomalies in the IoT on the TON-IoT set in Table 11.

Table 11 shows that the CNN-BMEGTO-KNN performed better than other studies by achieving 99.99% accuracy. These results prove that the proposed method has achieved superiority over hybrid methods such as GBM, XGBoost, and Ensemble regarding all four criteria. Swarm-NN strategy and DNN have achieved 99.5% and 99.92% accuracy, respectively, and performed better than other methods. Also, the proposed method is compared with hybrid feature selection methods such as XGBoost-SMOTE-Chi2 and Ensemble. The results show that the CNN-BMEGTO-KNN method exhibits high performance in all criteria due to better feature extraction, effective feature selection, and stronger classification.

## 5.4 Discussion and Evaluation

A general analysis of the results of the CNN-BMEGTO-KNN method and FECNNIoT deep network on the NSL-KDD and TON- IoT datasets is reflected in Fig. 11.

**Table 11** Comparison of the results of the CNN-BMEGTO-KNN method with other references and new methods on the TON- IoT dataset

| Refs. | Algorithm | Accuracy (%) | Precision (%) | Recall (%) | F1-score (%) |
|---|---|---|---|---|---|
| [38] | DNN | 99.92 | 99.90 | 99.87 | – |
| | RF | 99.98 | 99.95 | 99.99 | – |
| | GBM | 93.83 | 94.15 | 87.75 | – |
| [39] | XGBoost | 99.91 | 98.40 | 99.10 | 99.87 |
| | XGBoost-Chi2 | 98.30 | 98.40 | 96.70 | 97.50 |
| | XGBoost-SMOTE | 99.00 | 97.60 | 99.70 | 98.60 |
| | XGBoost-SMOTE- Chi2 | 98.20 | 95.40 | 99.60 | 99.75 |
| [40] | Swarm-NN strategy | 99.5 | – | – | – |
| [52] | DT | 88.67 | 84.82 | 82.29 | 83.54 |
| | NB | 74.13 | 60.98 | 71.44 | 65.77 |
| | RF | 93.94 | 86.88 | 97.32 | 91.81 |
| | Ensemble | 96.35 | 90.54 | 99.98 | 95.03 |
| [53] | DNRANN-1 | 99.15 | 99.23 | 99.07 | 99.27 |
| | DNRANN-2 | 99.20 | 98.91 | 98.86 | 98.88 |
| | DNRANN-3 | 98.17 | 98.89 | 98.83 | 98.86 |
| [54] | SATIDS | 96.35 | 98.4 | – | – |
| [55] | CNN-BMECapsA-RF | 99.99 | 99.99 | 99.99 | 99.99 |
| Proposed method | FECNNIoT | 99.83 | 99.81 | 99.82 | 99.81 |
| | CNN-BMEGTO-KNN | **99.99** | **99.99** | **99.99** | **99.99** |

Bold numbers indicate the improvement of the proposed method

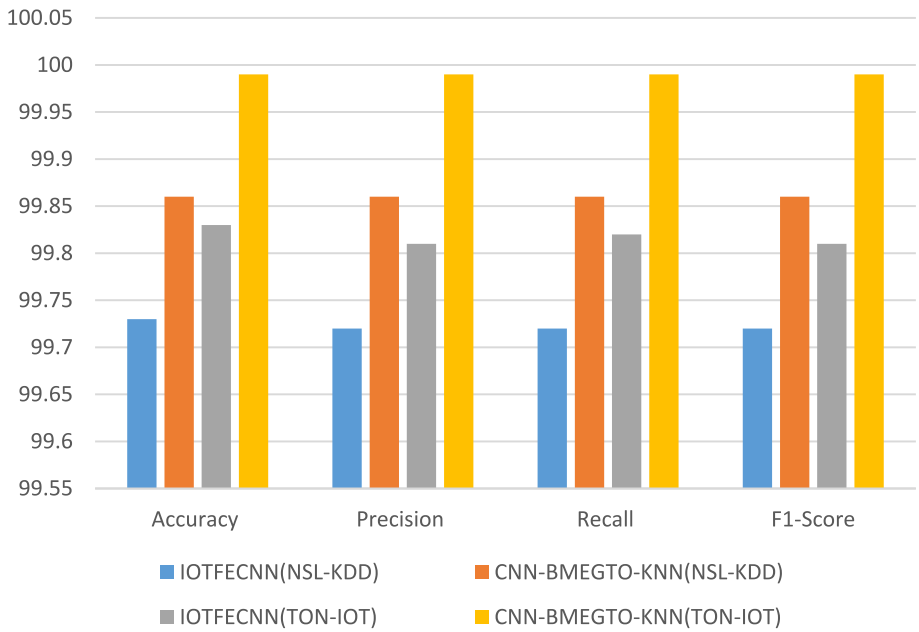



**Fig. 11** The results of the proposed hybrid method CNN-BMEGTO-KNN and the proposed deep network FECNNIoT on two different data sets

The FECNNIoT deep network has achieved an accuracy of 99.73 on the NSL-KDD dataset and has shown relatively good performance. But the CNN-BMEGTO-KNN method has achieved a high accuracy of 99.86 by selecting effective features, feature extraction, and classification based on the KNN algorithm. Also, FECNNIoT deep network and CNN-BMEGTO-KNN method have achieved 99.86% and 99.99% accuracy, respectively, on the TON-IoT dataset. Both methods have shown relatively favorable performance in all the examined criteria. This analysis proves that the FECNNIoT deep network can be considered a classification method and a feature extraction method in anomaly detection

datasets. The CNN-BMEGTO-KNN method has proven its superiority as a powerful method in anomaly detection.

## 6 Conclusion and Future Works

Convolutional neural networks are a well-known and efficient deep learning method due to their ability to extract features automatically. In this research, a convolutional neural network with hybrid layers called FECNNIoT was designed and implemented to extract high-level and low-level features to better detect anomalies on the Internet of Things. Also, a binary multi-objective enhanced gorilla troops optimizer called BMEGTO for efficient feature selection was developed. The combination of the FECNNIOT and BMEGTO method led to the introduction of a new hybrid method called CNN-BMEGTO-KNN. Finally, the proposed method was implemented and tested on two data sets, NSL-KDD and TON-IoT. The proposed method was compared with machine learning and deep learning methods regarding the accuracy, precision, recall, and F1 criteria. The experimental results show that the CNN-BMEGTO-KNN method has achieved 99.85% and 99.99% accuracy on the NSL-KDD and TON-IoT datasets, respectively, which is significantly superior compared to common deep learning methods such as LSTM and MLP. The CNN-BMEGTO-KNN method has shown high performance in detecting anomalies on the IoT in all criteria due to better feature extraction, effective feature selection, and stronger classification. Multi-class mode evaluation and attack type detection in IoT with the CNN-BMEGTO-KNN method and testing the proposed method on real online data will be our research direction in future works. Also, the design of other deep networks and the two-step detection of anomalies are left to the next research.

**Funding** Open access funding provided by the Scientific and Technological Research Council of Türkiye (TÜBİTAK).

**Data Availability** Data available on request from the authors.

## Declarations

**Conflict of interest** The authors declare no conflict of interest.

**Open Access** This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by/4.0/.

## References

1. Almiani, M., AbuGhazleh, A., Al-Rahayfeh, A., Atiewi, S., & Razaque, A. (2020). Deep recurrent neural network for IoT intrusion detection system. *Simulation Modelling Practice and Theory, 101*, 102031.
2. Cheng, B., Wang, M., Zhao, S., Zhai, Z., Zhu, D., & Chen, J. (2017). Situation-aware dynamic service coordination in an IoT environment. *IEEE/ACM Transactions On Networking, 25*(4), 2082–2095.
3. Jiang, H., Xiao, Z., Li, Z., Xu, J., Zeng, F., & Wang, D. (2020). An energy-efficient framework for internet of things underlaying heterogeneous small cell networks. *IEEE Transactions on Mobile Computing, 21*(1), 31–43.
4. Luo, J., Zhao, C., Chen, Q., & Li, G. (2022). Using deep belief network to construct the agricultural information system based on Internet of Things. *The Journal of Supercomputing, 78*(1), 379–405.
5. Zhang, X. M., Rao, Y., Xiong, Z. G., Deng, H. T., Xu, F., & Li, Y. (2023). Resource-constrained and socially selfish-based incentive algorithm for socially aware networks. *Journal of Signal Processing Systems, 95*(12), 1439–1453.
6. Saied, M., Guirguis, S., & Madbouly, M. (2024). Review of artificial intelligence for enhancing intrusion detection in the internet of things. *Engineering Applications of Artificial Intelligence, 2024*(127), 107231.
7. Li, S., Cao, Y., Liu, S., Lai, Y., Zhu, Y., & Ahmad, N. (2024). HDA-IDS: A hybrid DoS attacks intrusion detection system for IoT by using semi-supervised CL-GAN. *Expert Systems with Applications, 238*, 122198.
8. Jazebi, S. J., & Ghaffari, A. (2020). RISA: Routing scheme for Internet of Things using shuffled frog leaping optimization algorithm. *Journal of Ambient Intelligence and Humanized Computing, 11*(10), 4273–4283.
9. Seyfollahi, A., Abeshloo, H., & Ghaffari, A. (2022). Enhancing mobile crowdsensing in fog-based internet of things utilizing harris hawks optimization. *Journal of Ambient Intelligence and Humanized Computing, 13*, 1–16.
10. Seyfollahi, A., & Ghaffari, A. (2020). Reliable data dissemination for the Internet of Things using Harris hawks optimization. *Peer-to-Peer Networking and Applications, 13*(6), 1886–1902.
11. Seyfollahi, A., Taami, T., & Ghaffari, A. (2023). Towards developing a machine learning-metaheuristic-enhanced energy-sensitive routing framework for the internet of things. *Microprocessors and Microsystems, 96*, 104747.
12. Zheng, W., Deng, P., Gui, K., & Wu, X. (2023). An abstract syntax tree based static fuzzing mutation for vulnerability evolution analysis. *Information and Software Technology, 158*, 107194.
13. Choudhary, V., & Tanwar, S. (2024). Generation & evaluation of datasets for anomaly-based intrusion detection systems in IoT environments. *Multimedia Tools and Applications*. https://doi.org/10.1007/s11042-024-19066-2
14. Lin, Z., Wang, H., & Li, S. (2022). Pavement anomaly detection based on transformer and self-supervised learning. *Automation in Construction, 143*, 104544.
15. Wu, Z., Liu, G., Wu, J., & Tan, Y. (2023). Are neighbors alike? A semisupervised probabilistic collaborative learning model for online review spammers detection. *Information Systems Research*. https://doi.org/10.1287/isre.2022.0047
16. Daoud, M. S., Shehab, M., Abualigah, L., & Thanh, C. L. (2023). Hybrid modified chimp optimization algorithm and reinforcement learning for global numeric optimization. *Journal of Bionic Engineering, 20*(6), 2896–2915.
17. Yu, J., Lu, L., Chen, Y., Zhu, Y., & Kong, L. (2019). An indirect eavesdropping attack of keystrokes on touch screen through

acoustic sensing. *IEEE Transactions on Mobile Computing, 20*(2), 337–351.
18. Hasan, M., Islam, M. M., Zarif, M. I. I., & Hashem, M. (2019). Attack and anomaly detection in IoT sensors in IoT sites using machine learning approaches. *Internet of Things, 7*, 100059.
19. Yao, Y., Zhao, J., Li, Z., Cheng, X., & Wu, L. (2023). Jamming and eavesdropping defense scheme based on deep reinforcement learning in autonomous vehicle networks. *IEEE Transactions on Information Forensics and Security, 18*, 1211–1224.
20. Samadi, B. M., Ghaffari, A., Soleimanian, G. F., & Alemi, P. (2020). A wrapper-based feature selection for improving performance of intrusion detection systems. *International Journal of Communication Systems, 33*(12), e4434.
21. Hanafi, A. V., Ghaffari, A., Rezaei, H., Valipour, A., & Arasteh, B. (2023). Intrusion detection in Internet of things using improved binary golden jackal optimization algorithm and LSTM. *Cluster Computing, 27*, 1–18.
22. Bulla, C., & Birje, M. N. (2022). Anomaly detection in industrial IoT applications using deep learning approach. *Artificial Intelligence in Industrial Applications: Approaches to Solve the Intrinsic Industrial Optimization Problems, 25*, 127–147.
23. Cao, K., Wang, B., Ding, H., Lv, L., Dong, R., Cheng, T., & Gong, F. (2020). Improving physical layer security of uplink NOMA via energy harvesting jammers. *IEEE Transactions on Information Forensics and Security, 16*, 786–799.
24. Guo, R., Liu, H., & Liu, D. (2023). When deep learning-based soft sensors encounter reliability challenges: A practical knowledge-guided adversarial attack and its defense. *IEEE Transactions on Industrial Informatics*, 2702–2714. https://doi.org/10.1109/TII.2023.3297663
25. Yin, Y., Guo, Y., Su, Q., & Wang, Z. (2022). Task allocation of multiple unmanned aerial vehicles based on deep transfer reinforcement learning. *Drones, 6*(8), 215.
26. Zhang, J., Ren, J., Cui, Y., Fu, D., & Cong, J. (2024). Multi-USV task planning method based on improved deep reinforcement learning. *IEEE Internet of Things Journal*, 18549–18567. https://doi.org/10.1109/JIOT.2024.3363044
27. Abdollahzadeh, B., Soleimanian, G. F., & Mirjalili, S. (2021). Artificial gorilla troops optimizer: A new nature-inspired metaheuristic algorithm for global optimization problems. *International Journal of Intelligent Systems, 36*(10), 5887–5958.
28. Keserwani, P. K., Govil, M. C., Pilli, E. S., & Govil, P. (2021). A smart anomaly-based intrusion detection system for the Internet of Things (IoT) network using GWO–PSO–RF model. *Journal of Reliable Intelligent Environments, 7*(1), 3–21.
29. Mafarja, M., Heidari, A. A., Habib, M., Faris, H., Thaher, T., & Aljarah, I. (2020). Augmented whale feature selection for IoT attacks: Structure, analysis and applications. *Future Generation Computer Systems, 112*, 18–40.
30. Bhuvaneswari, A. N. G., & Selvakumar, S. (2020). Anomaly detection framework for Internet of things traffic using vector convolutional deep learning approach in fog environment. *Future Generation Computer Systems, 113*, 255–265.
31. Ullah, I., & Mahmoud, Q. H. (2021). Design and development of a deep learning-based model for anomaly detection in IoT networks. *IEEE Access, 9*, 103906–103926.
32. de Antonio Cristiano, S., Westphall, C. B., Machado, R. B., Sobral, J. B. M., & Dos Santos Vieira, G. (2020). Hybrid approach to intrusion detection in fog-based IoT environments. *Computer Networks, 180*, 107417.
33. Wei, W., Chen, S., Lin, Q., Ji, J., & Chen, J. (2020). A multi-objective immune algorithm for intrusion feature selection. *Applied Soft Computing, 95*, 106522.
34. Roopak, M., Tian, G. Y., & Chambers, J. (2020). Multi-objective-based feature selection for DDoS attack detection in IoT networks. *IET Networks, 9*(3), 120–127.
35. Alazzam, H., Sharieh, A., & Sabri, K. E. (2020). A feature selection algorithm for intrusion detection system based on pigeon inspired optimizer. *Expert systems with applications, 148*, 113249.
36. Fatani, A., Abd, E., & M., Dahou, A., Al-Qaness, M.A., & Lu, S. (2021). IoT intrusion detection system using deep learning and enhanced transient search optimization. *IEEE Access, 9*, 123448–123464.
37. Kan, X., Fan, Y., Fang, Z., Cao, L., Xiong, N. N., Yang, D., & Li, X. (2021). A novel IoT network intrusion detection approach based on adaptive particle swarm optimization convolutional neural network. *Information Sciences, 568*, 147–162.
38. Moustafa, N. (2021). A new distributed architecture for evaluating AI-based security systems at the edge: Network TON_IoT datasets. *Sustainable Cities and Society, 72*, 102994.
39. Gad, A. R., Nashat, A. A., & Barkat, T. M. (2021). (2021). Intrusion detection system using machine learning for vehicular ad hoc networks based on ToN-IoT dataset. *IEEE Access, 9*, 142206–142217.
40. Nandy, S., Adhikari, M., Khan, M. A., Menon, V. G., & Verma, S. (2021). An intrusion detection mechanism for secured IoMT framework based on swarm-neural network. *IEEE Journal of Biomedical and Health Informatics, 26*(5), 1969–1976.
41. Negandhi, P., Trivedi, Y., & Mangrulkar, R. (2018). Intrusion detection system using random forest on the NSL-KDD dataset. *Emerging Research in Computing, Information, Communication and Applications: ERCICA, 2*, 519–531.
42. Khanday, S. A., Fatima, H., & Rakesh, N. (2023). Implementation of intrusion detection model for DDoS attacks in Lightweight IoT Networks. *Expert Systems with Applications, 215*, 119330.
43. Abdelaziz, M., Al-qaness, M. A., Dahou, A., Ibrahim, R. A., & Abdel-Latif, A. A. (2023). Intrusion detection approach for cloud and IoT environments using deep learning and Capuchin Search Algorithm. *Advances in Engineering Software, 176*, 103402.
44. Ding, Y., Zhang, W., Zhou, X., Liao, Q., Luo, Q., & Ni, L. M. (2020). FraudTrip: Taxi fraudulent trip detection from corresponding trajectories. *IEEE Internet of Things Journal, 8*(16), 12505–12517.
45. Cao, B., Zhao, J., Yang, P., Gu, Y., Muhammad, K., Rodrigues, J. J., & Dealbuquerque, V. H. C. (2019). Multiobjective 3-D topology optimization of next-generation wireless data center network. *IEEE Transactions on Industrial Informatics, 16*(5), 3597–3605.
46. Hussien, A. G., Hassanien, A. E., Houssein, E. H., Bhattacharyya, S., & Amin, M. (2019). S-shaped binary whale optimization algorithm for feature selection. In S. Bhattacharyya, A. Mukherjee, H. Bhaumik, S. Das, & K. Yoshida (Eds.), *Recent trends in signal and image processing: ISSIP* (pp. 79–87). Springer Singapore.
47. Altman, N. S. (1992). An introduction to kernel and nearest-neighbor nonparametric regression. *The American Statistician, 46*(3), 175–185.
48. Qian, L., Zheng, Y., Li, L., Ma, Y., Zhou, C., & Zhang, D. (2022). A new method of inland water ship trajectory prediction based on long short-term memory network optimized by genetic algorithm. *Applied Sciences, 12*(8), 4073.
49. Hosseini, S., & Sardo, S. R. (2022). Network intrusion detection based on deep learning method in internet of thing. *Journal of Reliable Intelligent Environments, 9*(2), 147–159.
50. Jothi, B., & Pushpalatha, M. (2021). WILS-TRS—A novel optimized deep learning based intrusion detection framework for IoT networks. *Personal and Ubiquitous Computin, 27*, 1–17.
51. Diro, A. A., & Chilamkurti, N. (2018). Distributed attack detection scheme using deep learning approach for Internet of Things. *Future Generation Computer Systems, 82*, 761–768.
52. Kumar, P., Gupta, G. P., & Tripathi, R. (2021). An ensemble learning and fog-cloud architecture-driven cyber-attack detection

framework for IoMT networks. *Computer Communications, 166*, 110–124.
53. Latif, S., Huma, Z., Jamal, S. S., Ahmed, F., Ahmad, J., Zahid, A., Dashtipour, K., Aftab, M. U., Ahmad, M., & Abbasi, Q. H. (2021). Intrusion detection framework for the internet of things using a dense random neural network. *IEEE Transactions on Industrial Informatics, 18*, 6435–6444.
54. Elsayed, R. A., Hamada, R. A., Abdalla, M. I., & Elsaid, S. A. (2023). Securing IoT and SDN systems using deep-learning based automatic intrusion detection. *Ain Shams Engineering Journal, 14*(10), 102211.
55. Asgharzadeh, H., Ghaffari, A., Masdari, M., & Soleimanian, G. F. (2023). Anomaly-based intrusion detection system in the Internet of Things using a convolutional neural network and multi-objective enhanced Capuchin Search Algorithm. *Journal of Parallel and Distributed Computing, 175*, 1–21.

**Publisher's note** Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.